Advanced Higher Project

Lucy's Big Quiz

St Kentigern's Academy

Contents

Analysis

1.1 Description of the Problem

Description of the Problem:

I am going to code a one-player quiz game for my Advanced Higher project. It will primarily be coded in Java and integrated with SQL.

The aim of the game is to answer questions which progressively get harder, advancing with each correct answer. If the player gets a question wrong they will be unable to continue and the game will automatically stop.

The game will be multiple choice and the player can choose between answers A, B, C and D. I intend to implement clickable buttons which will make it easier for the user to input their answer. The players nickname and the number of questions they got correct will be displayed on the leaderboard, ranking from highest-scoring to lowest-scoring.

In order to meet SQA advanced higher requirements my project will include:

- A 2D array to create a leaderboard
- Integration between my software program and my SQL database
- Using SQL to create my database and leaderboard table
- Using SQL to insert the data
- A bubble sort to sort the leaderboard from highest to lowest score
- Validation for all inputs

1.2 Constraints

Constraints:

Technical Constraints:

Software to code the program in Java and SQL
Netbeans and Repl for code
Windows 10
I will be using the school desktop for this program. The spec of this desktop:
- RAM: 8GB (7.88GBG usable)
- Processor: Intel(R) Core(TM) i5-6500T CPU @ 2.50GHz 2.50GHz

- I will also be using my home laptop. The spec of this laptop:
  - RAM: 32.0 GB (31.4 GB usable)
  - Processor: AMD Ryzen 7 4800H with Radeon Graphics 2.90 GHz

Time Constraints:

- The project must be completed by Friday 7th April 2023, a time limit set by the SQA

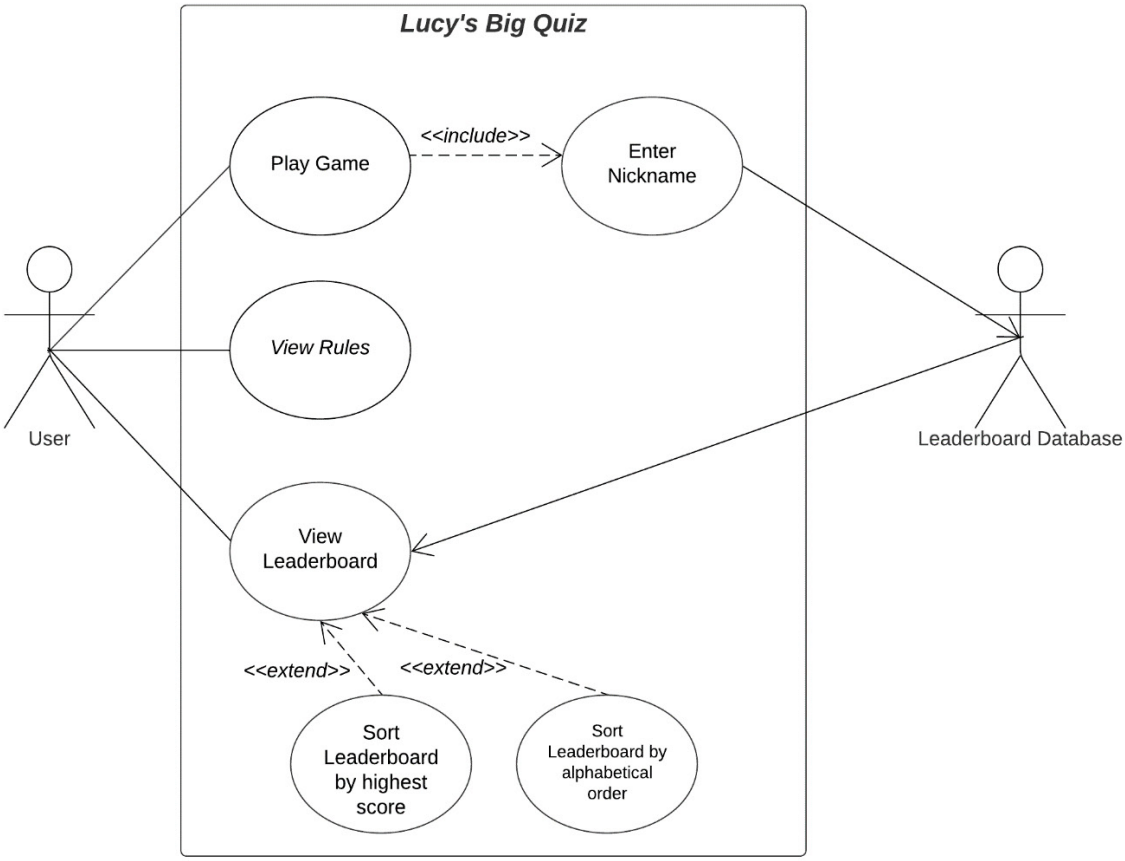
Legal Constraints:

- This project must follow GDPR Regulations and I must make sure it does not contravene any legal constraints

Economic Constraints:

- This project will not require any costs to complete

1.3 UML Use Case Diagram



1.4 Requirements Specification

Functional Requirements:

Inputs:

(1a) The user can start playing the game by inputting 1 from the keyboard
(1b) The user can view the rules by inputting 2 from the keyboard
(1c) The user can view the leaderboard by inputting 3 from the keyboard
(1d) The user can exit the program by inputting 4 from the keyboard
(1e) The user will enter their username
(1f) The user will input their answer by selecting one of the buttons shown to them
(1g) When provided an option to go back to the home page the user should input 1 from the keyboard

Processes:

- **(2a)** The program will read in the questions from the data file
- **(2b)** The program will randomise one question from the first set of 10 questions which are of the easiest difficulty. As the user progresses the program will move on to the next set of 10 questions and randomly pick one from there
- **(2c)** When the user selects the button the program will check if the answer they selected matches up with the question. If it does the program moves on to the next question and if it is wrong then the game stops and the user is shown their score
- **(2d)** The program will count up how many questions the user got correct
- **(2e) The user's username and score will be stored in a table in an external database**
- **(2f)** A leaderboard will be created containing the top 5 highest scores
- **(2g)** A bubble sort will sort the leaderboard into the highest scores
- **(2h)** When provided an option to go back to the home page the program should validate whether the user inputs 1
- **(2i)** On the home page the program should validate the users input and check if its 1, 2, 3 or 4
- **(2j)** The program should validate the users nickname to make sure it is no more than 10 characters

Outputs:

- **(3a)** The program will display a message at the start of the game with the name of the game and the options the user can pick
- **(3b)** If the user **selects 'rules' then the program will display the rules**
- **(3c)** The program will ask the user for a nickname of 10 characters
- **(3d) The program will output text which lets the user know what question they're on, the actual** question and then the four answers the user can choose from
- **(3e)** The program will output 4 buttons labelled A, B, C and D which the four possible answer are assigned to. This stops the users submitting an invalid answer
- **(3f)** The program will output the closing message containing the users score
- **(3g)** The program will output the sorted leaderboard, showing the players position, nickname and score

End-user Requirements:

From interviewing users, I have discovered these are my end-user requirements I will include in my program:

- **(4a)** There should be an opening message at the beginning
- **(4b)** The user should enter a nickname rather than their real name
- **(4c)** The user should have 15 questions to answer
- **(4d)** The user should have 4 different answer options provided to them
- **(4e)** The user should have buttons to input their answer instead of having to type it themselves which will validate the input
- **(4f)** The user should lose the game instantly if they get an answer wrong
- **(4g)** There should be a closing message at the end of the program
- **(4h)** The user should be able to view the top 5 names and scores in a leaderboard

1.5 Project Plan

Topic	Task	Begin	Targeted Finish	Resources
Analysis	Description of the Problem	29/8/22	5/9/22	Microsoft Word
	Constraints	12/9/22	15/9/22	Microsoft Word
	UML Use Case Diagram	19/9/22	23/9/22	Lucidchart
	Requirement specification	5/9/22	12/9/22	Microsoft Word
	Functional & End-User Requirements	12/9/22	19/9/22	Microsoft Word
	Project Plan	23/9/22	30/9/22	Microsoft Word
	Gantt Chart	23/9/22	30/9/22	Microsoft Excel
Design	Top Level Algorithm & Dataflow	3/10/22	9/10/22	Microsoft Word
	Integration Design	3/10/22	9/10/22	Microsoft Word
	Data Dictionary	9/10/22	16/10/22	Microsoft Word
	SQL Query Design	9/10/22	16/10/22	Microsoft Word
	User Interface	16/10/22	23/10/22	Microsoft Paint
Implementation	Opening message	23/10/22	31/10/22	Repl.it/Netbeans IDE
	Display Rules	23/10/22	31/10/22	Repl.it/Netbeans IDE
	Return Home	1/11/22	7/11/22	Repl.it/Netbeans IDE
	Read in questions	1/11/22	7/11/22	Repl.it/Netbeans IDE
	Play game and get users nickname	7/11/22	14/11/22	Repl.it/Netbeans IDE
	Randomise questions which increase in difficulty	14/11/22	21/11/22	Repl.it/Netbeans IDE
	Research and implement new skill (buttons)	21/11/22	5/12/22	Repl.it/Netbeans IDE/Internet
	Closing message	5/12/22	12/12/22	Repl.it/Netbeans IDE
	Get leaderboard from database	12/12/22	19/12/22	Netbeans IDE/ Xampp and MySQL

	Insert new data into leaderboard and bubble sort	19/12/22	23/12/22	Netbeans IDE/ Xampp/MySQL
	Display leaderboard	9/1/23	16/1/23	Netbeans IDE/ Xampp/MySQL
	Implemented UI	16/1/23	23/1/23	Netbeans IDE
	Description of New Skills	16/1/23	23/1/23	Microsoft Word/Internet
	On-going Testing	23/1/23	30/1/23	Microsoft Word/ Repl.it/Netbeans IDE
Testing	Test Plan	30/1/23	6/2/23	Microsoft Word
	Functional Requirements Testing	6/2/23	13/2/23	Microsoft Word/Netbeans IDE
	End-User Requirements Testing	13/2/23	20/2/23	Microsoft Word/Netbeans IDE
	Persona Testing	20/2/23	27/2/23	Microsoft Word/Netbeans IDE
Evaluation	Fitness for Purpose	27/2/23	6/3/23	Microsoft Word/Netbeans IDE
	Maintainability and Robustness	6/3/23	13/3/23	Microsoft Word/Netbeans IDE

Design

2.1 Top Level Algorithm & Dataflow

1. Declare global variables
2. Display Opening Message OUT: vOption
3. Play Game IN: questionsTable, aLeaderboard
4. Display Rules
5. Display Leaderboard IN: aLeaderboard OUT: aLeaderboard

2.1 Ask user to choose one of the 4 options
2.2 Return vOption OUT: vOption

2 Validate the user's option by showing an error method if their option is not one of the 4 they are supposed to choose from ***Validation***

3.1 If vOption equals 1 call readQuestions method IN: questionsTable
3.2 Call playGame method IN: questionsTable, aLeaderboard
3.3 Call displayQuestion method IN: questionsTable, aLeaderboard

3 Open fileReader
3 Loop for number of questions in the questionsTable array of records
3 Read data in from the CSV file into the questionsTable array of records
3 End loop
3 Close fileReader
3 Displays title and prompts user to enter a nickname with a max of 10 characters
3 Get playerName from keyboard
3 While declare int length initialised to length of playerName is more than 10, print out a message stating the nickname is too long and to try again ***validation***
3 Get playerName from keyboard
3 End while
3 Use a Java Switch Statement
3 <If questionNum equals 1 then case 1 carries out vRndNum equals a random number between 1 and 10>

***Continue until if questionNum equals 15 then case 15 carries out vRndNum equals a random number between 141 and 150**
3.3.3 <Display a text area with A, B, C and D buttons> ***New skill learned***
3.3.4 When button A is clicked, if questionNum is less than 15 do
3.3.5 Else add 1 to numCorrect and call closingMessage method
3.3.6 When button B is clicked, if questionNum is less than 15 do
3.3.7 Else add 1 to numCorrect and call closingMessage method
3.3.8 When button C is clicked, if questionNum is less than 15 do
3.3.9 Else add 1 to numCorrect and call closingMessage method
3.3.10 When button D is clicked, if questionNum is less than 15 do
3.3.11 Else add 1 to numCorrect and call closingMessage method

3 When closingMessage method is called do IN: aLeaderboard
3 Display message saying “you scored” and displaying numCorrect
3.3.14 closingMessage method then calls readLeaderboard, sortLeaderboard and displayLeaderboard methods
These are shown below

3.3.4.1 If answerA at vRndNum in questionsTable is equal to correctAns at vRndNum in questionsTable, add 1 to questionNum and add 1 to numCorrect and call displayQuestion method again
3.3.4.2 Else display a message saying incorrect answer and call closingMessage method and hide buttons
3.3.6.1 If answerB at vRndNum in questionsTable is equal to correctAns at vRndNum in questionsTable, add 1 to questionNum and add 1 to numCorrect and call displayQuestion method again
3.3.6.2 Else display a message saying incorrect answer and call closingMessage method and hide buttons
3.3.8.1 If answerC at vRndNum in questionsTable is equal to correctAns at vRndNum in questionsTable, add 1 to questionNum and add 1 to numCorrect and call displayQuestion method again
3.3.8.2 Else display a message saying incorrect answer and call closingMessage method and hide buttons
3.3.10.1 If answerD at vRndNum in questionsTable is equal to correctAns at vRndNum in questionsTable, add 1 to questionNum and add 1 to numCorrect and call displayQuestion method again
3.3.10.2 Else display a message saying incorrect answer and call closingMessage method and hide buttons

4.1 If vOption equals 2 call rules method
4.2 <Display rules>
4.3 Call returnHome

4.3.1 Display message asking user to input 1 to return to home page
4.3.2 Declare int vReturnHome and initialise it to users input from keyboard
4.3.3 If vReturnHome equals 1 call processGame
4.3.4 While vReturnHome does not equal 1 display an error message and ask the user to try again and get vReturnHome from keyboard

5.1 If vOption equals 3 call readLeaderboard IN: aLeaderboard
5.2 call sortLeaderboard IN: aLeaderboard
5.3 displayLeaderboard IN: aLeaderboard
5 Connect to lucysbigquiz database using JDBC
5 Read database to a 2D array called aLeaderboard
5 Close connection
5 Assign aLeaderboard row 5 column 1 as playerName
5 Assign aLeaderboard row 5 column 2 as numCorrect
5 Declare int n and initialise to 6
5 Declare boolean swapped and initialise to true
5 While swapped is true and n is more than 0 do
5 Swapped is false
5 For index from 0 to n-1 do
5 Declare int vNum1 and initialise to row index and column 2 in aLeaderboard

5.2.9 Declare int vNum2 and initialise to row index+1 and column 2 in aLeaderboard
5.2.10 If vNum1 is less than vNum2 do
5.2.11 Declare String temp is equal to row index and column 2 in
5.2.12 aLeaderboard
Set row index and column 2 in aLeaderboard to row index+1 and column 2 in aLeaderboard
5.2.13 Set row index+1 and column 2 in aLeaderboard to temp
5.2.14 Set temp to row index and column 1 in aLeaderboard
5.2.15 Set row index and column 1 in aLeaderboard to row index+1 and column 1 in aLeaderboard
5.2.16 Set row index+1 and column 1 in aLeaderboard to temp
5.2.17 Set swapped to true
5.2.18 End if
5.2.19 End for
5.2.20 Set n to n-1
5.2.21 End while

5.3.1 Loop 5 times
5.3.2 Display row index and column 0 in aLeaderboard
5.3.3 Display row index and column 1 in aLeaderboard
5.3.4 Display row index and column 2 in aLeaderboard
5.3.5 End loop

2.2 Integration Design

Read in data from database:

1. Assign server connection variables
2. Open connection to lucysbigquiz database using JDBC
3. Using a loop read in lucysbigquiz database to a 2D array called aLeaderboard
4. End loop
5. Close connection to lucysbigquiz database

Bubble sort data:

1. Assign aLeaderboard row 5 column 1 as playerName
2. Assign aLeaderboard row 5 column 2 as numCorrect
3. Declare int n and initialise to 6
4. Declare boolean swapped and initialise to true
5. While swapped is true and n is more than 0 do
6. Swapped is false
7. For index from 0 to n-1 do
8. Declare int vNum1 and initialise to row index and column 2 in aLeaderboard
9. Declare int vNum2 and initialise to row index+1 and column 2 in aLeaderboard
10. If vNum1 is less than vNum2 do
11. Declare String temp is equal to row index and column 2 in aLeaderboard
12. Set row index and column 2 in aLeaderboard to row index+1 and column 2 in aLeaderboard
13. Set row index+1 and column 2 in aLeaderboard to temp
14. Set temp to row index and column 1 in aLeaderboard
15. Set row index and column 1 in aLeaderboard to row index+1 and column 1 in aLeaderboard
16. Set row index+1 and column 1 in aLeaderboard to temp
17. Set swapped to true
18. End if
19. End for
20. Set n to n-1
21. End while

Display data from database:

1. Loop for 5 times
2. Output the players position, playerName and numCorrect

2.3 Data Dictionary

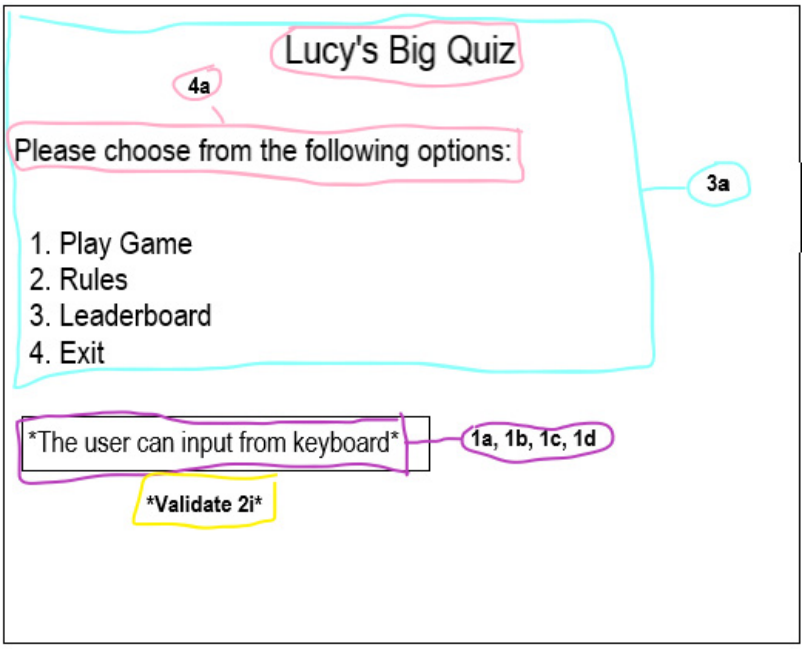
Attribute Name	Key	Type	Size	Required	Validation
id	PK	integer		yes	Auto increment
name		varchar	10	yes	
score		integer		yes	

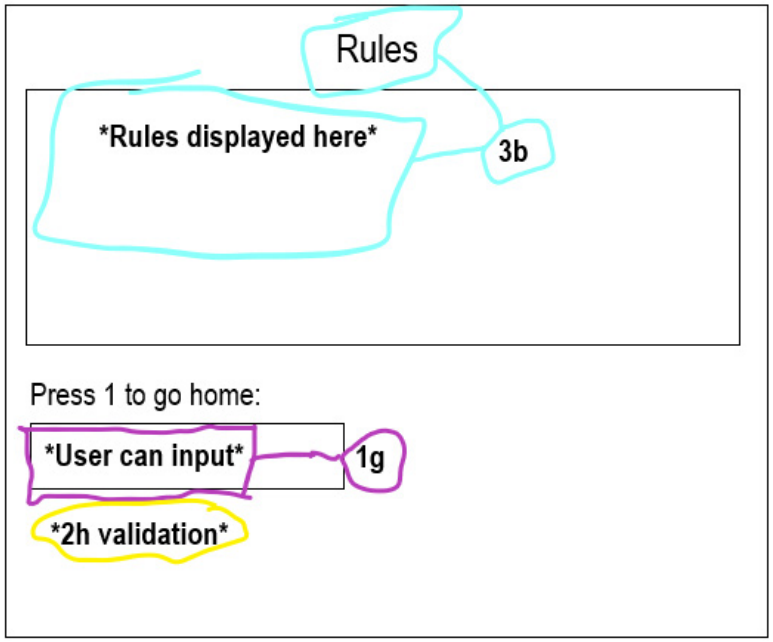
2.4 SQL Query Design

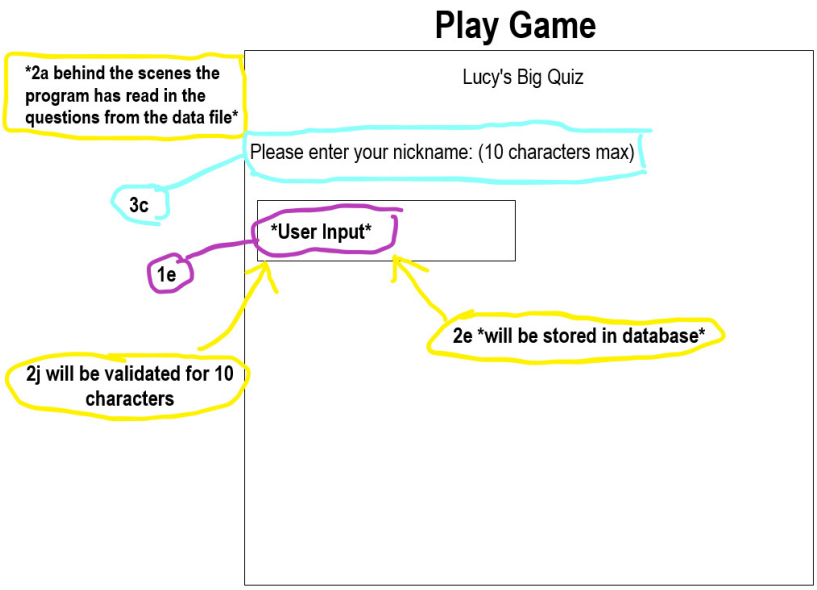
Select query which will display the top 5 players' positions, name and score from lucysbigquiz database

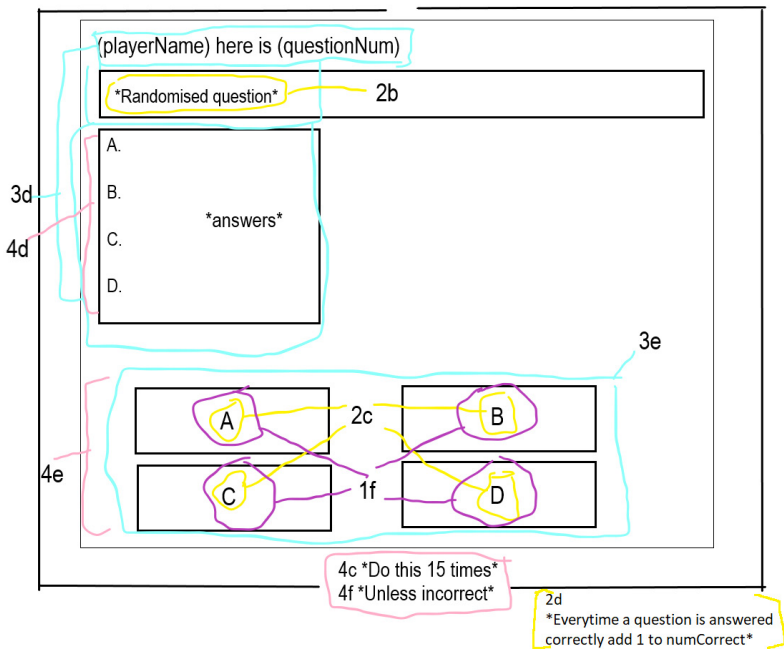
Field(s)/ calculation(s)	id, name, score
Table(s) query(-ies)	leaderboard
Search criteria	
Grouping	
Having	
Sort order	

2.5 User Interface

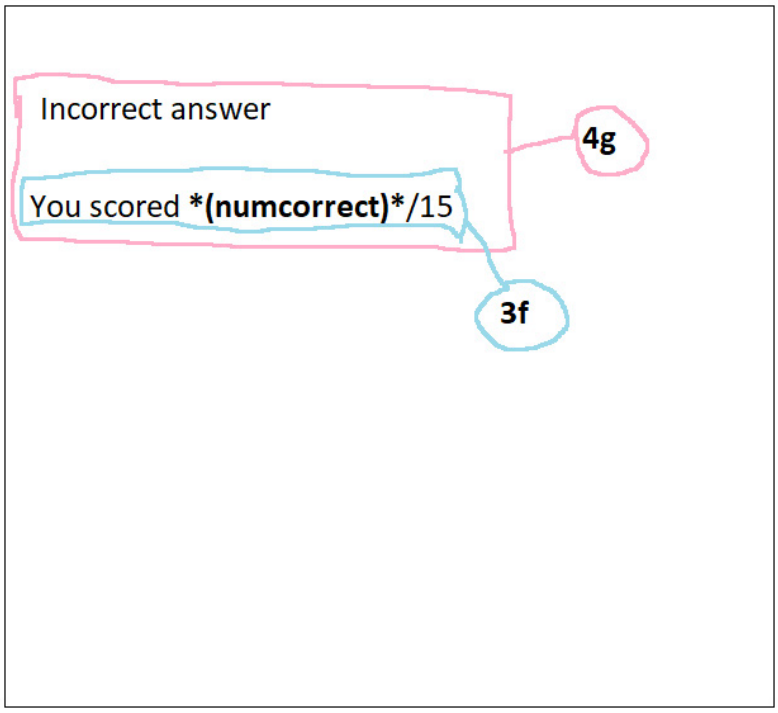


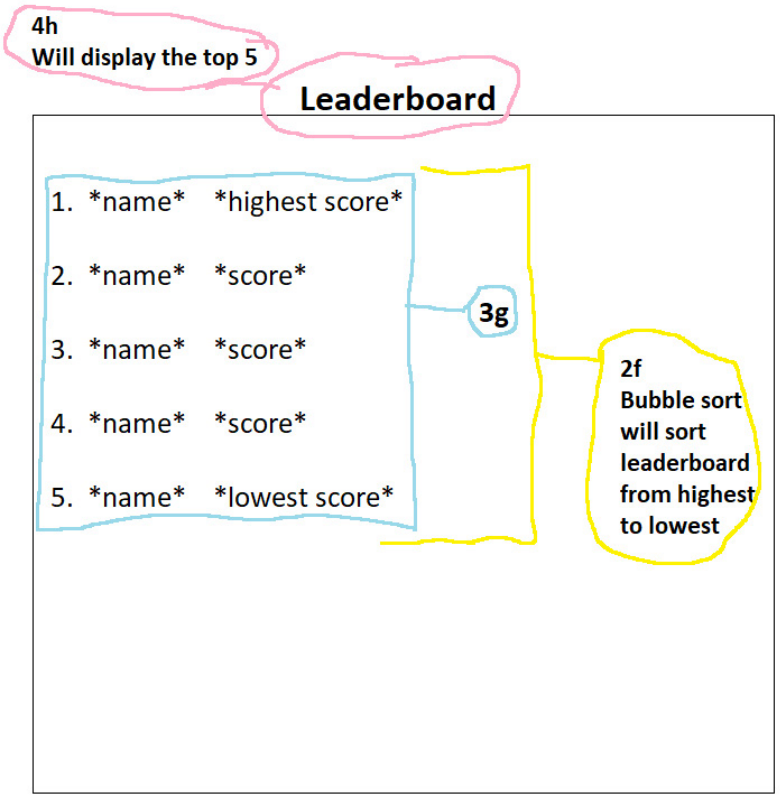






Closing Message





Implementation

3.1 Java Code

```
package lucysbigquiz;


import java.io.*;
import java.sql.*;
import java.util.*;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;


public class LucysBigQuiz {


    class questions {
        String question =  ;
        String answerA =  ;
        String answerB =  ;
        String answerC =  ;
        String answerD =  ;
        String correctAns =  ;
    } // end class questions


    /**
     * @param args the command line arguments
```

```
 * @throws java.io.IOException
 */

public static void main(String[] args) throws IOException {
  LucysBigQuiz myProg = new LucysBigQuiz();
  myProg.processGame();
}

Scanner keyboard = new Scanner(System.in);
//variables declared
String playerName =  ;
int questionNum = 1;
int vRndNum = 0;
int numCorrect = 0; public void processGame()

throws IOException {

  int vOption = 0;
  //2d array and array of records declared
  String[][] aLeaderboard = new String[6][3];
  questions[] questionsTable = new questions[150];

  //Will display the opening message
  vOption = displayOpeningMsg();
```

```
    //validating user input (2i)
     while ((vOption < 1) || (vOption > 4) ) {
         System.out.println("Error. Please enter a valid option: ");
         vOption = keyboard.nextInt();
         keyboard.nextLine();
      }


    switch (vOption) {
      case 1: //if the user inputs 1 then the program will read in the questions, then play the game (1a)
          readQuestions(questionsTable);
          playGame(questionsTable, aLeaderboard);
         break;
      case 2: //if the user inputs 2 then the program will show the rules (1b)
          rules();
         break;
case 3: //if the user inputs 3 then the program will read in the leaderboard data, put it through a bubble sort and then display it (1c)
          readLeaderboard(aLeaderboard);
          sortLeaderboard(aLeaderboard);
          displayLeaderboard(aLeaderboard);
         break;
      case 4: //if the user inputs 4 they will exit the program (1d)
          System.exit(0);
          break;
     } //end switch case
   } // end processGame
```

```
    public void closingMessage(String[][] aLeaderboard) {


// this clears the screen
     System.out.print("\033[H\033[2J");
     System.out.flush();


       for (int i = 0; i < 37; i++) {


        System.out.print("\u001B[35m" + "*"); // purple
        System.out.print("\033[0;34m" + "*"); // blue
       }


     System.out.println();
     System.out.println();
     System.out.println("\u001B[37m" + /* bold */"\u001b[1m" + "                    Lucy's Big Quiz" + /* un-bold
*/"\u001b[0m");
     System.out.println();


       for (int i = 0; i < 37; i++) {
        System.out.print("\u001B[35m" + "*"); // purple
        System.out.print("\033[0;34m" + "*"); // blue
       }
     //displays closing message (2c)(3f)(4g)
```

```
  System.out.println("You scored " + numCorrect + "/15");

  //reads, sorts and then displays the leaderboard
  readLeaderboard(aLeaderboard);
  sortLeaderboard(aLeaderboard);
  displayLeaderboard(aLeaderboard);

  //exits program
  System.exit(0);
} //end closingMessage

/**
 *
 * @param questionsTable
 * @param aLeaderboard
 */

public void displayQuestion(questions[] questionsTable, String[][] aLeaderboard) {
    //Based on the question the user is on, it will randomise a question of the level difficulty (2b)
    switch (questionNum) {
      case 1:
        vRndNum = (int) (Math.random() * 10);
        break;
      case 2:
        vRndNum = (int) (Math.random() * 10) + 10;
```

```
  break;
case 3:
  vRndNum = (int) (Math.random() * 10) + 20;
  break;
case 4:
  vRndNum = (int) (Math.random() * 10) + 30;
  break;
case 5:
  vRndNum = (int) (Math.random() * 10) + 40;
  break;
case 6:
  vRndNum = (int) (Math.random() * 10) + 50;
  break;
case 7:
  vRndNum = (int) (Math.random() * 10) + 60;
  break;
case 8:
  vRndNum = (int) (Math.random() * 10) + 70;
  break;
case 9:
  vRndNum = (int) (Math.random() * 10) + 80;
  break;
case 10:
  vRndNum = (int) (Math.random() * 10) + 90;
  break;
```

```
        case 11:
          vRndNum = (int) (Math.random() * 10) + 100;
          break;
        case 12:
          vRndNum = (int) (Math.random() * 10) + 110;
          break;
        case 13:
          vRndNum = (int) (Math.random() * 10) + 120;
          break;
        case 14:
          vRndNum = (int) (Math.random() * 10) + 130;
          break;
        case 15:
          vRndNum = (int) (Math.random() * 10) + 140;
          break;
      } //end switch case (4c)

    JFrame frame = new JFrame("Lucy's Big Quiz");

    //text for the questions here (3d)(4d)

JTextArea textArea = new JTextArea(playerName + " here is question " + questionNum + "\n" +
questionsTable[vRndNum].question + "\n" + "A) " + questionsTable[vRndNum].answerA + "\n" + "B) " +
questionsTable[vRndNum].answerB + "\n" + "C) " + questionsTable[vRndNum].answerC + "\n" + "D) " +
questionsTable[vRndNum].answerD, 5, 20);

    JScrollPane scroll = new JScrollPane(textArea);
```

// **create a new button with label (3e)(4e)**

JButton buttonA = new JButton("A");

JButton buttonB = new JButton("B");

JButton buttonC = new JButton("C");

JButton buttonD = new JButton("D");

// **set bounds for the button (x cord, y - coord, width, height)**

buttonA.setBounds(0, 160, 150, 30);

buttonB.setBounds(160, 160, 150, 30);

buttonC.setBounds(0, 200, 150, 30);

buttonD.setBounds(160, 200, 150, 30);

// **scrollable text area**

scroll.setSize(800, 300);

scroll.getViewport().setViewPosition(new java.awt.Point(350, 150));

// **add the button to the frame**

frame.add(buttonA);

```
frame.add(buttonB);

frame.add(buttonC);

frame.add(buttonD);

// add the text frame
frame.add(scroll);

// set the size and visibility of the frame
frame.setSize(550, 300);

frame.setLayout(null);

frame.setVisible(true);

//(1f)
  buttonA.addActionListener(e -> {
   //when the button is clicked, answerA is compared to correctAns
   frame.setVisible(false);

     //if the question number is less than 15
     if (questionNum < 15) {
```

//check if answer A of the question the player is on is equal to the correct answer for the same question (2c)

```
    if (questionsTable[vRndNum].answerA.equals(questionsTable[vRndNum].correctAns) ) {

     //increase question number
     questionNum++;

     //increase the number of questions the user has answered correctly (2d)
     numCorrect++;

     //call the displayQuestion method again to move on to the next question (2c)
     displayQuestion(questionsTable, aLeaderboard);
    }
    else { //if the user answered incorrectly (2c)(4f)
     System.out.println("Incorrect answer. Better luck next time");

     //displays the closing message
     closingMessage(aLeaderboard);

     //stops the buttons from showing
     frame.setVisible(false);
    }
}
//if the question number is NOT less than 15
else {
 //increase the number of questions the user has answered correctly (2d)
 numCorrect++;
```

```
        //displays the closing message
        closingMessage(aLeaderboard);
      }
    }); //end buttonA


    buttonB.addActionListener(e -> {
     //when the button is clicked, answerB is compared to correctAns
     frame.setVisible(false);


       //if the question number is less than 15
       if (questionNum < 15) {
//check if answer B of the question the player is on is equal to the correct answer for the same question (2c)
          if (questionsTable[vRndNum].answerB.equals(questionsTable[vRndNum].correctAns) ) {


            //increase question number
            questionNum++;


            //increase the number of questions the user has answered correctly (2d)
            numCorrect++;


            //call the displayQuestion method again to move on to the next question
            displayQuestion(questionsTable, aLeaderboard);
```

```
        }
        else { //if the user answered incorrectly (2c)(4f)
         System.out.println("Incorrect answer. Better luck next time");

         //displays the closing message
         closingMessage(aLeaderboard);

         //stops the buttons from showing
         frame.setVisible(false);
        }
      }
      else { //if the question number is NOT less than 15

       //increase the number of questions the user has answered correctly (2d)
       numCorrect++;

       //displays the closing message
       closingMessage(aLeaderboard);
      }
});//end buttonB

buttonC.addActionListener(e -> {
 //when the button is clicked, answerC is compared to correctAns
 frame.setVisible(false);
```

```
        //if the question number is less than 15
        if (questionNum < 15) {
//check if answer C of the question the player is on is equal to the correct answer for the same question (2c)
            if (questionsTable[vRndNum].answerC.equals(questionsTable[vRndNum].correctAns) ) {

             //increase question number
             questionNum++;

             //increase the number of questions the user has answered correctly (2d)
             numCorrect++;

             //call the displayQuestion method again to move on to the next question
             displayQuestion(questionsTable, aLeaderboard);
            }
            else { //if the user answered incorrectly (2c)(4f)
             System.out.println("Incorrect answer. Better luck next time");

             //displays the closing message
             closingMessage(aLeaderboard);

             //stops the buttons from showing
             frame.setVisible(false);
            }
        }
        else { //if the question number is NOT less than 15
```

```
        //increase the number of questions the user has answered correctly (2d)
        numCorrect++;

        //displays the closing message
        closingMessage(aLeaderboard);
       }
    }); //end buttonC

    buttonD.addActionListener(e -> {
     //when the button is clicked, answerD is compared to correctAns
     frame.setVisible(false);

       //if the question number is less than 15
       if (questionNum < 15) {
//check if answer D of the question the player is on is equal to the correct answer for the same question (2c)
          if (questionsTable[vRndNum].answerD.equals(questionsTable[vRndNum].correctAns) ) {

           //increase question number
           questionNum++;

           //increase the number of questions the user has answered correctly (2d)
           numCorrect++;

           //call the displayQuestion method again to move on to the next question
```

```
            displayQuestion(questionsTable, aLeaderboard);
          }
          else { //if the user answered incorrectly (2c)(4f)
            System.out.println("Incorrect answer. Better luck next time");

            //displays the closing message
            closingMessage(aLeaderboard);

            //stops the buttons from showing
            frame.setVisible(false);
          }
        }
        else { //if the question number is NOT less than 15

          //increase the number of questions the user has answered correctly (2d)
          numCorrect++;

          //displays the closing message
          closingMessage(aLeaderboard);
        }
    }); //end buttonD
} //end displayQuestion
```

```
public void playGame(questions[] questionsTable, String[][] aLeaderboard) throws IOException {
  // this clears the screen
  System.out.print("\033[H\033[2J");
  System.out.flush();

  for (int i = 0; i < 37; i++) {
    System.out.print("\u001B[35m" + "*"); // purple
    System.out.print("\033[0;34m" + "*"); // blue
  }

  System.out.println();
  System.out.println();
  System.out.println("\u001B[37m" + /* bold */"\u001b[1m" + "                    Lucy's Big Quiz"
      + /* un-bold */"\u001b[0m");
  System.out.println();

  for (int i = 0; i < 37; i++) {
    System.out.print("\u001B[35m" + "*"); // purple
    System.out.print("\033[0;34m" + "*"); // blue
  }

  System.out.println();
  System.out.println("Please enter your nickname: (10 characters max)");

  //(3c)(4b) playerName = keyboard.nextLine(); //(1e)
```

```
  int length = playerName.length();

  //validates player's nickname so they cant enter a long nickname over 10 characters (2j)
     while (length > 10) {
        System.out.println("Your nickname is too long. Please enter your nickname: (10 characters
        max)"); playerName = keyboard.nextLine();

        length = playerName.length();
     }
  System.out.print("\033[H\033[2J");
  System.out.flush();

  displayQuestion(questionsTable, aLeaderboard);
} //end playGame



public void readQuestions(questions[] questionsTable) throws IOException {
    //reads in the questions from the csv file and assigns it to the record (2a)
     try (Scanner fileReader = new Scanner(new File("D:\\data.csv"))) {
        fileReader.useDelimiter(",|\n");

        for (int i = 0; i < questionsTable.length; i++) {
           questionsTable[i] = new questions();
           questionsTable[i].question = fileReader.next();
```

```
            questionsTable[i].answerA = fileReader.next();
            questionsTable[i].answerB = fileReader.next();
            questionsTable[i].answerC = fileReader.next();
            questionsTable[i].answerD = fileReader.next();
            questionsTable[i].correctAns = fileReader.next();
        }
    }
} // end readQuestions



public void returnHome() throws IOException {
 //allows the user to go back to the home page
 System.out.println();
 System.out.println("Enter 1 to return to home page.");

 int vReturnHome = keyboard.nextInt(); //(1g)
 keyboard.nextLine();

 while (vReturnHome != 1) { //validation (2h)
       System.out.println("Wrong input. Please enter 1");
        vReturnHome = keyboard.nextInt();
        keyboard.nextLine();
    }
```

```
      if (vReturnHome == 1) {
       processGame();
      }
  } // end returnHome



public void rules() throws IOException {
    //displays the rules for the player (3b)

    // this clears the screen
   System.out.print("\033[H\033[2J");
   System.out.flush();

     for (int i = 0; i < 37; i++) {
       System.out.print("\u001B[35m" + "*"); // purple
       System.out.print("\033[0;34m" + "*"); // blue
     }

   System.out.println();
   System.out.println("\u001B[37m" /* white */ + /* bold */"\u001b[1m" + "              Rules" + /* un-bold */"\u001b[0m");
   System.out.println();

     for (int i = 0; i < 37; i++) {
       System.out.print("\u001B[35m" + "*"); // purple
```

```
        System.out.print("\033[0;34m" + "*"); // blue
      }


    System.out.println();
System.out.println("Lucy's Big Quiz is a series of multiple-choice questions that each have four possible answers (A,B,C,D). There will be buttons with each letter and you have to press the one you think is the correct answer.");
    System.out.println();
    System.out.println("You must answer 15 of these questions correctly, one at a time, in order to win.");
    System.out.println();
    System.out.println("As soon as you answer a question incorrectly, the game is over.");
    System.out.println();
    returnHome();
   } // end rules


   //(3a)(4a)
   public int displayOpeningMsg() throws IOException {

    int vOption = 0;

    // this clears the screen
    System.out.print("\033[H\033[2J");
    System.out.flush();

      for (int i = 0; i < 37; i++) {
```

```
        System.out.print("\u001B[35m" + "*"); // purple
        System.out.print("\033[0;34m" + "*"); // blue
       }

    System.out.println();
    System.out.println();
    System.out.println("\u001B[37m" + /* bold */"\u001b[1m" + "                    Lucy's Big Quiz" + /* un-bold */"\u001b[0m");
    System.out.println();

       for (int i = 0; i < 37; i++) {
        System.out.print("\u001B[35m" + "*"); // purple
        System.out.print("\033[0;34m" + "*"); // blue
       }

    System.out.println();
    System.out.println("Please choose one of the following options: ");
    System.out.println();
    System.out.println("1. Play Game");
    System.out.println("2. Rules");
    System.out.println("3. Leaderboard");
    System.out.println("4. Exit");

    vOption = keyboard.nextInt();
    keyboard.nextLine();
```

```
    return vOption;

  } // end displayOpeningMsg

} //end LucysBigQuiz
```

3.2 Integration Code

```
public void displayLeaderboard(String[][] aLeaderboard) {

  //This will display the databases 5 rows (3g)(4h)
    for(int i = 0; i < 5; i++) {
     System.out.printf("%-20s", aLeaderboard[i][0]);

     System.out.printf("%-20s", aLeaderboard[i][1]);

     System.out.printf("%-20s", aLeaderboard[i][2]);

     System.out.println();
    }
 }//end displayLeaderboard



 public void sortLeaderboard(String[][] aLeaderboard) {
  //assigns players nickname and score to database (2e)
  aLeaderboard[5][1] = playerName;
  aLeaderboard[5][2] = Integer.toString(numCorrect);

  //bubble sort will sort the name and score based on the score (2g)
```

```
int n = 6; // DECLARE n INITIALLY 6

boolean swapped = true; // DECLARE swapped INITIALLY TRUE

  while ((swapped) && (n > 0)) { // WHILE swapped AND n > 0

   swapped = false; // SET swapped TO False

    for (int i = 0; i < n - 1; i++) { // FOR i = 0 to n-1 DO

     int vNum1 = Integer.parseInt(aLeaderboard[i][2]);
     int vNum2 = Integer.parseInt(aLeaderboard[i+1][2]);

      if (vNum1 < vNum2) { // IF vNum1 > vNum2 THEN
       String temp = aLeaderboard[i][2]; // SET temp TO aLeaderboard[i][2]
       aLeaderboard[i][2] = aLeaderboard[i + 1][2]; // SET aLeaderboard[i][2] TO aLeaderboard[i +
1][2]
       aLeaderboard[i + 1][2] = temp; // SET aLeaderboard[i + 1][2] TO temp
       temp = aLeaderboard[i][1]; // SET temp TO aLeaderboard[i][1]
       aLeaderboard[i][1] = aLeaderboard[i + 1][1]; // SET aLeaderboard[i][1] TO aLeaderboard[i +
1][1]
       aLeaderboard[i + 1][1] = temp; // SET aLeaderboard[i + 1][1] TO temp
       swapped = true; // SET swapped TO TRUE
      } // END IF
    } // END FOR
    n--; // SET n TO n - 1
```

```
    } // END WHILE
  } //end sortLeaderboard


  //(2f)
  public void readLeaderboard(String[][] aLeaderboard) {
   //read in the 5 rows from the database and put into a 2d array
   Connection con = null;



      try {

       con=DriverManager.getConnection("jdbc:mysql://localhost:3306/lucysbigquiz","root", );

Statement stmt=con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_READ_ONLY);

       ResultSet rs=stmt.executeQuery("select * from leaderboard");

       int numCols;

       numCols = rs.getMetaData().getColumnCount();

       int row = 0; // set the row counter to zero
```

```
    rs.beforeFirst(); // Set cursor to before the first record


      while (rs.next()) { // iterate through the rows
         for (int col = 0; col < numCols; col++) { // iterate through the columns
          aLeaderboard[row][col] = rs.getString(col + 1);
         }
       row++;
      }
     con.close();
    }


    catch(SQLException e) {
      System.err.println(e);
    }
} //end readLeaderboard
```

3.3 Implemented UI

Home Page

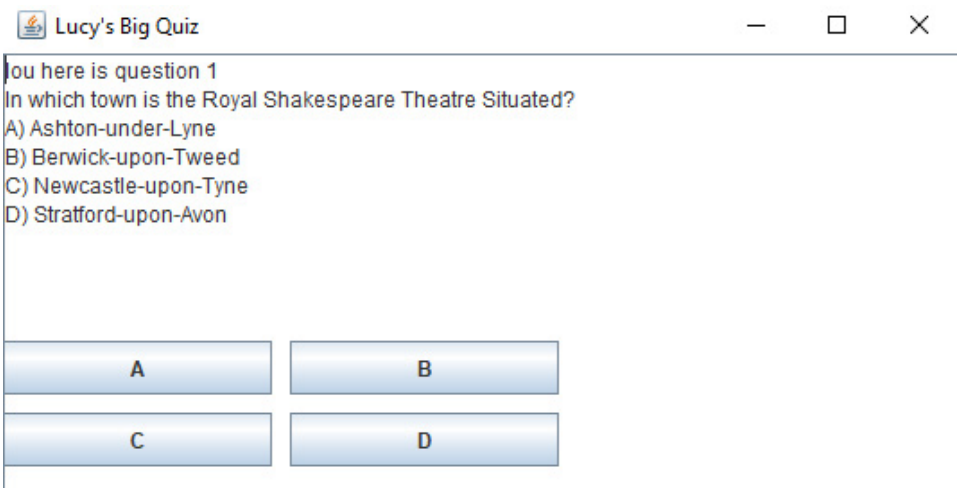
```
********************************************************************************

                        Lucy's Big Quiz

********************************************************************************
Please choose one of the following options:

1. Play Game
2. Rules
3. Leaderboard
4. Exit
```

Rules Page

```
********************************************************************
              Rules
********************************************************************
Lucy's Big Quiz is a series of multiple-choice questions that each have four possible answers (A,B,C,D). There will be buttons with each letter and you have to press the one you think is the correct answer.

You must answer 15 of these questions correctly, one at a time, in order to win.

As soon as you answer a question incorrectly, the game is over.


Enter 1 to return to home page.
```

Play Game

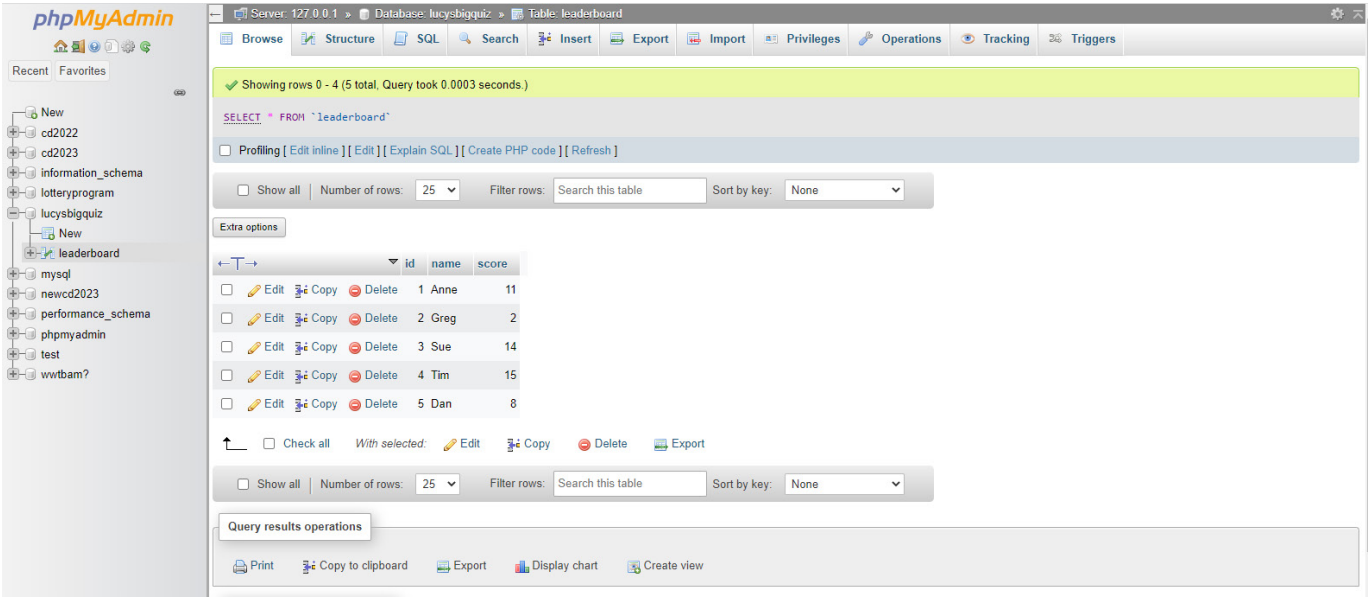
```
********************************************************************************

                        Lucy's Big Quiz

********************************************************************************
Please enter your nickname: (10 characters max)
```



Closing Message

```
Incorrect answer. Better luck next time
You scored 2/15
```

Leaderboard before sort



Leaderboard before user is added

1	Tim	15
2	Sue	14
3	Anne	11
4	Dan	8
5	Greg	2

Leaderboard after user is added

1	Tim	15
2	Sue	14
3	Anne	11
4	Dan	8
5	lou	3

3.4 Description of New Skills

Java JButton

I wanted to implement buttons into my project so that it limited the chances of the user inputting a wrong answer. In order to do this, I had to research different methods for buttons. Buttons were not a part of the course content and I had to learn this by myself. I took this following code from several websites and messed around with it to suit what I needed with 4 buttons:

```
JTextArea area=new JTextArea("Welcome to javatpoint");

  JScrollPane scrollableTextArea = new JScrollPane(textArea);

JFrame f=new JFrame("Button Example");
JButton b=new JButton(new ImageIcon("D:\\icon.png"));
b.setBounds(100,100,100, 40);
f.add(b);
f.setSize(300,400);
f.setLayout(null);
f.setVisible(true);
```

```
JFrame frame = new JFrame("Lucy's Big Quiz");

//text for the questions here (3d)(4d)
JTextArea textArea = new JTextArea(playerName + " here is question " + questionNum + "\n"
        + questionsTable[vRndNum].question + "\n" + "A) " + questionsTable[vRndNum].answerA + "\n" + "B) " + questionsTable[vRndNum].answerB + "\n" + "C) "
        + questionsTable[vRndNum].answerC + "\n" + "D) " + questionsTable[vRndNum].answerD, 5, 20);

JScrollPane scroll = new JScrollPane(textArea);

// create a new button with label (3e)(4e)
JButton buttonA = new JButton("A");

JButton buttonB = new JButton("B");

JButton buttonC = new JButton("C");

JButton buttonD = new JButton("D");

// set bounds for the button (x cord, y - coord, width, height)
buttonA.setBounds(0, 160, 150, 30);

buttonB.setBounds(160, 160, 150, 30);

buttonC.setBounds(0, 200, 150, 30);

buttonD.setBounds(160, 200, 150, 30);

// scrollable text area
scroll.setSize(800, 300);

scroll.getViewport().setViewPosition(new java.awt.Point(350, 150));

// add the button to the frame
frame.add(buttonA);

frame.add(buttonB);
```

```
frame.add(buttonC);

frame.add(buttonD);

// add the text frame
frame.add(scroll);

// set the size and visibility of the frame
frame.setSize(550, 300);

frame.setLayout(null);

frame.setVisible(true);

//(1f)
   buttonA.addActionListener(e -> {
    //when the button is clicked, answerA is compared to correctAns
    frame.setVisible(false);

       //if the question number is less than 15
       if (questionNum < 15) {
           //check if answer A of the question the player is on is equal to the correct answer for the same question (2c)
           if (questionsTable[vRndNum].answerA.equals(questionsTable[vRndNum].correctAns) ) {

            //increase question number
            questionNum++;

            //increase the number of questions the user has answered correctly (2d)
            numCorrect++;

            //call the displayQuestion method again to move on to the next question (2c)
            displayQuestion(questionsTable, aLeaderboard);
           }
           else { //if the user answered incorrectly (2c)(4f)
            System.out.println("Incorrect answer. Better luck next time");

            //displays the closing message
```

(buttonB, buttonC and buttonD have the same code as buttonA)

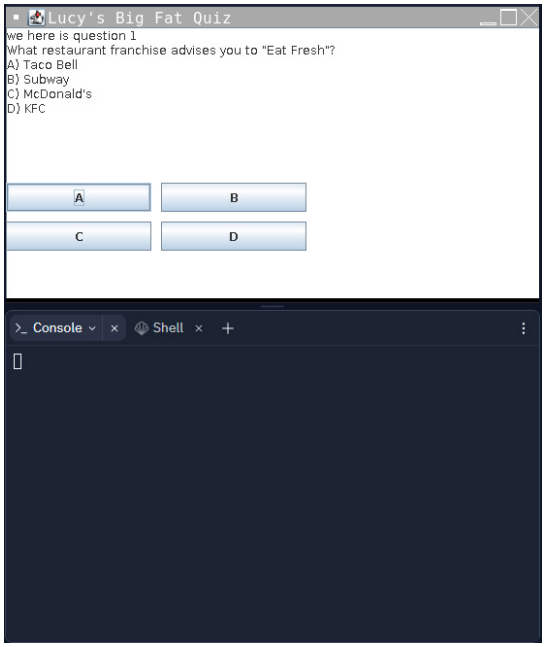
Links

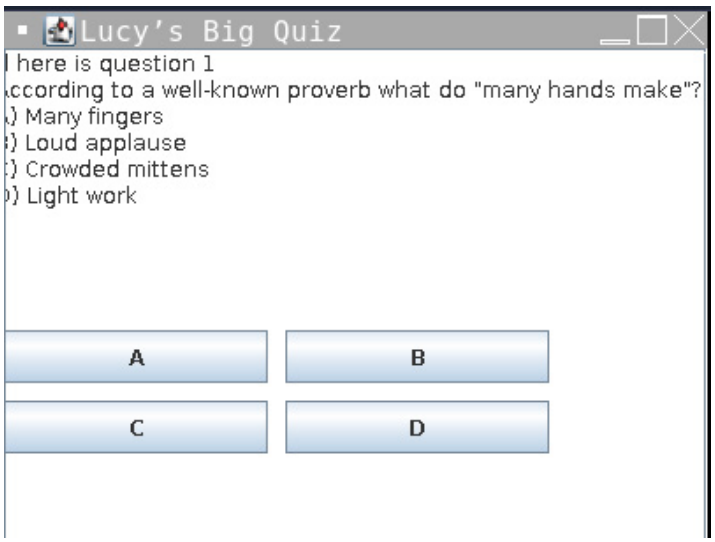
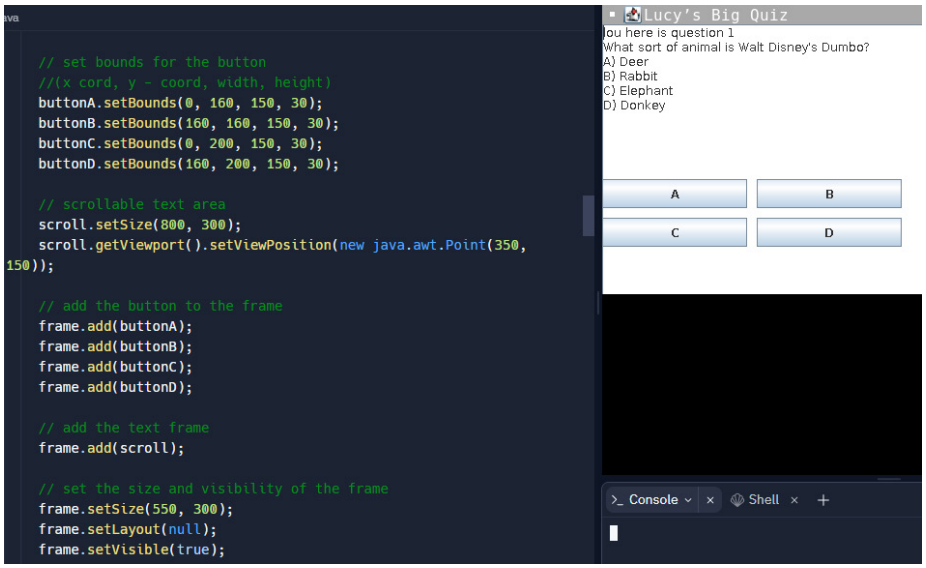
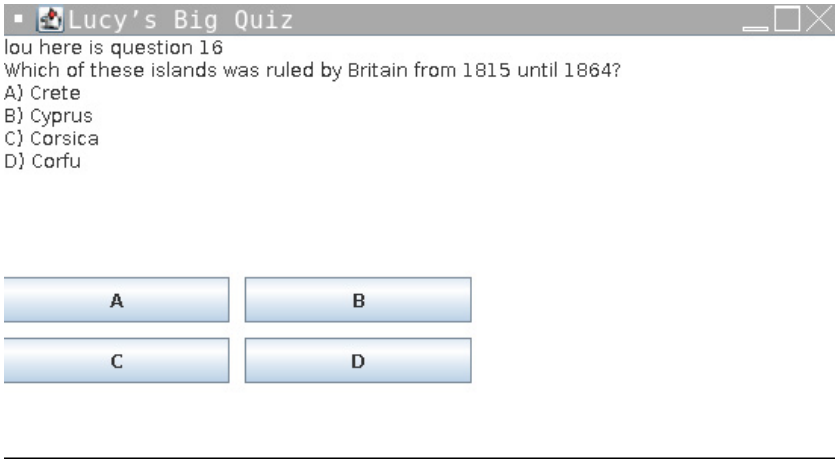
https://www.javatpoint.com/java-jbutton
https://docs.oracle.com/javase/tutorial/uiswing/events/actionlistener.html
https://www.javatpoint.com/java-jtextarea
https://www.javatpoint.com/java-jscrollpane

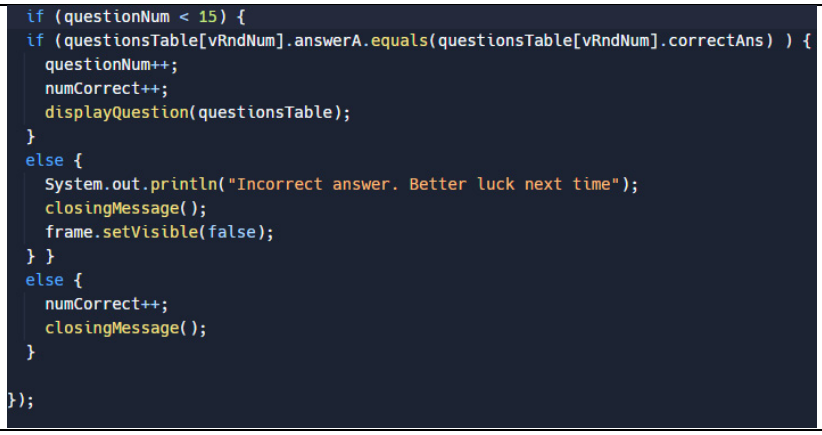
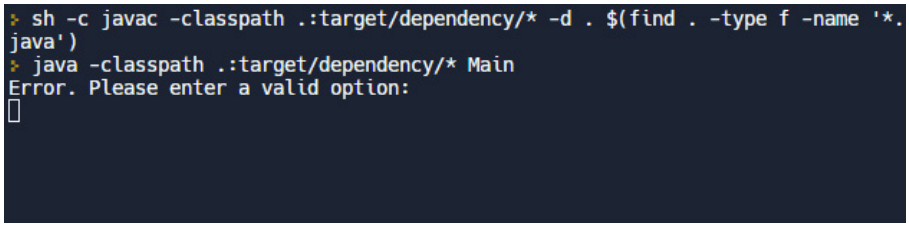
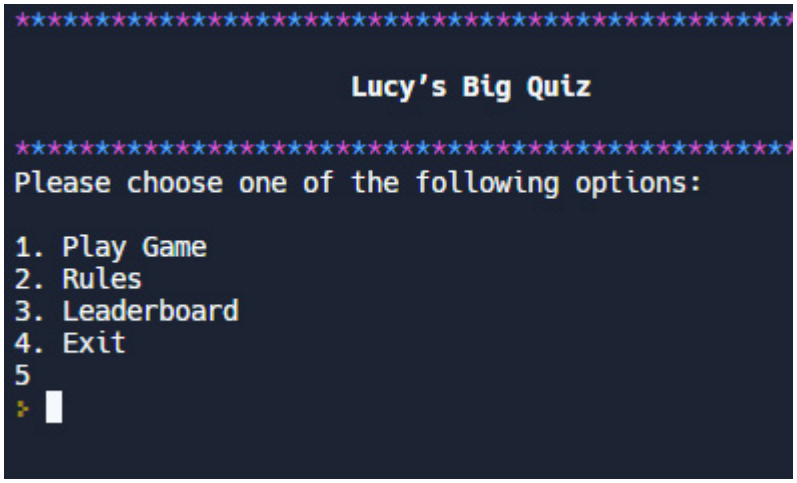
3.5 On-going Testing

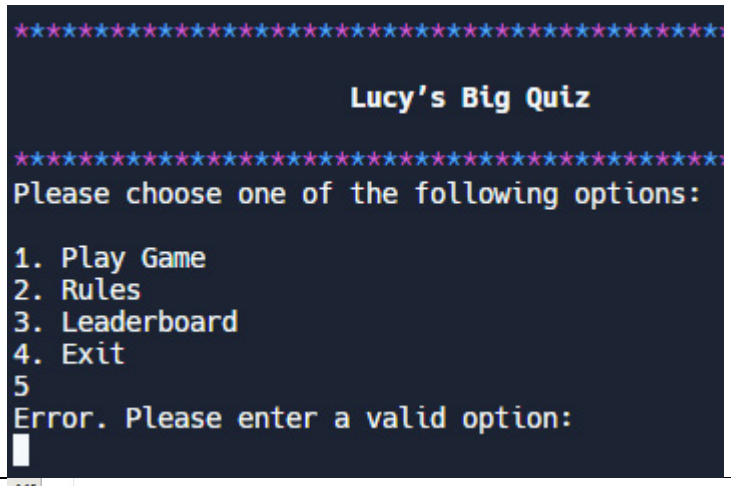
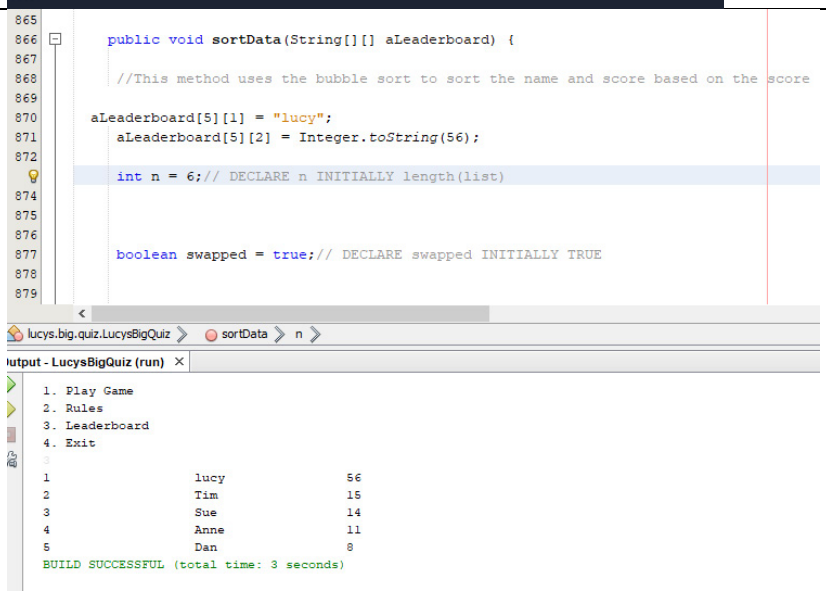
Test No.	Issue	How I fixed it My	Screenshots

1.	I couldn't get my program to read in the questions into an array of records	program required a throws IOException so I put one in where required	Before:
			`sh -c javac -classpath .:target/dependency/* -d . $(find . -type f -name '*.java')`
			`./Game.java:135: error: unreported exception IOException; must be caught or declared to be thrown`
			`readQuestions(questionsTable);`
			`1 error`
			`exit status 1`
			After:
			`public void readQuestions(questions[] questionsTable) throws IOException {`
			`Scanner fileReader = new Scanner(new File("data.csv"));`

2.	I was trying to get my buttons to compare the answer they are associated with with the correct answer, however nothing was working inside my if statements	My problem was actually with my data file as I was using the wrong one which had the correct answers stored as A, B, C or D instead of the actual answer	Before:
			 
			
			After:
			`buttonA.addActionListener(e -> {`
			`//when the button is clicked, answerA is compared to correctAns`
			`if (questionsTable[vRndNum].answerA.equals(questionsTable[vRndNum].correctAns)) {`
			`questionNum++;`
			`displayQuestion(questionsTable);`
			`}`
			`else {`
			`System.out.println("Incorrect answer");`
			`System.exit(0);`
			`}`
			`});`

3.	My JFrame and its contents kept clipping the sides of the screen which meant that some of the informati on wasn't visible	I had to mess about with the sizes to get what I wanted and for all the information to be visible	Before:  After: 
4.	My program was not stopping at question 15 and instead would keep increasin g in question number	I added code that stated that if the questionNu m was less than 15 then it could continue to compare the button the user clicked to the correct answer	Before:  After:

			
5.	When trying to implement my validation for the home page, the first thing that would appear was this error message instead of the actual home page	I moved where my validation is supposed to be which helped my home page appear first	Before: 
6.	My validation was again not working as when I entered an invalid number it wasn't displaying the error message I wanted it to or letting the	I had to move the while loop to before my if's and else's instead of after them	Before: 

	player enter a new option		After: 
7.	I wanted to test that my database worked and sorted my data	I inserted my own data to just make sure it worked	

8.	When I wanted to put the players name and score into my leaderboard it would not let me put in their score and I	I had to convert the score to an Int so that it was compatible with my 2D array and could be added to my database	Before:
			`public void sortLeaderboard(String[][] aLeaderboard) {`
			`//assigns players nickname and score to database (2e)`
			`aLeaderboard[5][1] = playerName;`
			`aLeaderboard[5][2] = numCorrect;`
			
			After:
			`public void sortLeaderboard(String[][] aLeaderboard) {`
			`//assigns players nickname and score to database (2e)`
			`aLeaderboard[5][1] = playerName;`
			`aLeaderboard[5][2] = Integer.toString(numCorrect);`

Testing

4.1 Test Plan

Functional Requirements

I will individually test each of the functional requirements, separated into their inputs, processes and outputs categories, and then record these results in a table. This table will include what the functional requirement is, a description of the test, any errors that are discovered and solutions where required. I will then provide evidence of these tests by taking screenshots.

End-User Requirements

I will individually test each of the end-user requirements and then record these results in a table. This table will include the end-user requirement and will have the same contents as the functional requirement table: a description of the test, any errors that are discovered and solutions where required and then evidence of these tests through screenshots.

Input Validations

I will include my input validation testing, which includes normal and exceptional data in my functional and end-user requirement tables when they come up and will clearly mark where input validation is being discussed. This will help to test the robustness of the quiz game.

Test Cases and Personas

I will have actors who will interact with the program. To do this they will be given several tasks to complete which will purposefully test different aspects of the program, such as if it meets the requirements and if it is easy for users to understand and use. I will display the evidence from this in a table.

Personas

Name: Roza Bilinska
Age: 69
Tech Experience: Emailing and phoning/texting family and friends

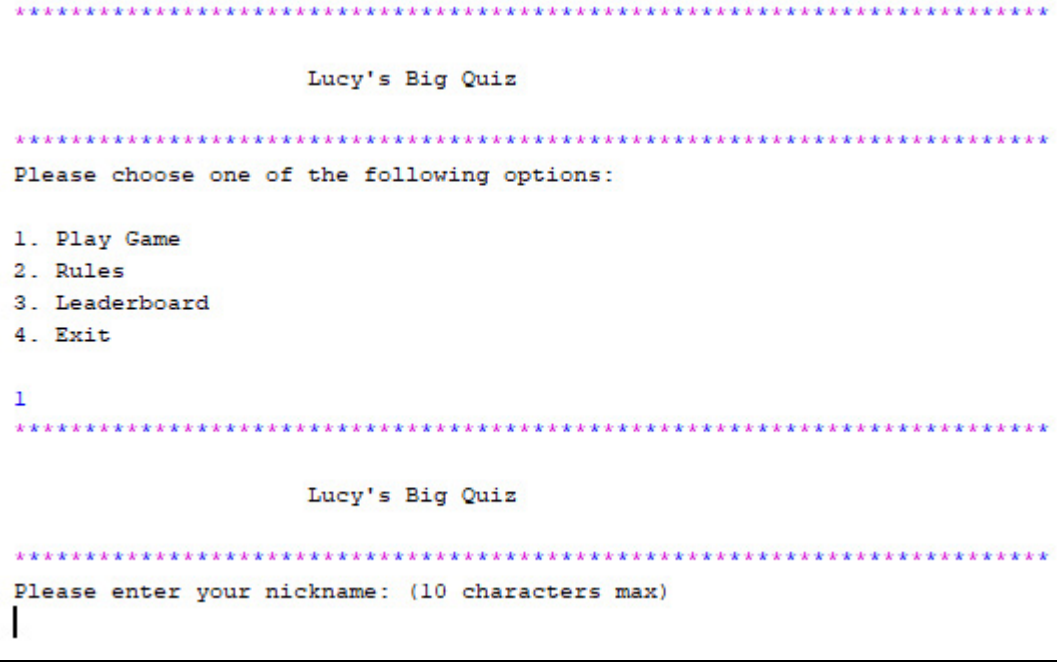
Name: Ellis Ramsay
Age: 21
Tech Experience: Social media, Gaming and frequently uses the internet

4.2 Functional Requirements Testing

Functional Requirements	description of the test	discovered errors	solution if required
Inputs:			
1. The user can start playing the game by inputting 1 from the keyboard (1a)	I will input 1 and it will take me to play the game	none	none
2. The user can view the rules by inputting 2 from the keyboard (1b)	I will input 2 and it will take me to the rules	none	none
3. The user can view the leaderboard by inputting 3 from the keyboard (1c)	I will input 3 and it will take me to view the leaderboard	none	none
4. The user can exit the program by inputting 4 from the keyboard (1d)	I will input 4 and it will exit the program	none	none
5. The user will enter their username (1e)	I will input a nickname and the game should continue	none	none
6. The user will input their answer by selecting one of the buttons shown to them (1f)	I will click a button and it should do something (either move on to the next question or show me the closing message)	none	none
7. When provided an option to go back to the home page the user should input 1 from the keyboard (1g)	I will input 1 and I should be taken back to the home page	none	none
Processes:			
8. The program will read in the questions from the data file (2a)	I will go through questions and see if the questions from my data file are appearing	none	none
9. The program will randomise one question from the first set of 10 questions which are of the easiest difficulty. As the user progresses the program will move on to the next set of 10 questions and randomly pick one from there (2b)	I will play the game several times and see if the questions are different the majority of each time	none	none
10. When the user selects the button the program will check if the answer they selected matches up	I will answer one question correct to see if it moves on to the next question like it's	none	none

with the question. If it does the program moves on to the next question and if it is wrong then the game stops and the user is shown their score (2c)	supposed to. I will then answer a question incorrectly to see if it shows me the closing message like it's supposed to		
11. The program will count up how many questions the user got correct (2d)	I will keep track of how many questions I answer correctly and see if the program displays the correct amount	none	none
12. The user's username and score will be stored in a table in an external database (2e)	I will check the database	none	none
13. A leaderboard will be created containing the top 5 highest scores (2f)	I will view the leaderboard and see if the top 5 highest scores are there	none	none
14. A bubble sort will sort the leaderboard into the highest scores (2g)	I will play the game and see if the program bubble sorts me into the correct place	none	none
15. When provided an option to go back to the home page the program should validate whether the user inputs 1 (2h)	I will enter exceptional inputs and then the correct one to see if the validation works	none	none
16. On the home page the program should validate the users input and check if its 1, 2, 3 or 4 (2i)	I will enter exceptional inputs and then a valid one to see if the validation works	none	none
17. The program should validate the users nickname to make sure it is no more than 10 characters (2j)	I will enter a nickname longer than 10 characters and I should get an error message	none	none
Outputs:			
18. The program will display a message at the start of the game with the name of the game and the options the user can pick (3a)	I will start the program and screenshot the first thing the program does	none	none
19. If the user selects 'rules' then the program will display the rules	I will select the rules and see what displays	none	none
20. The program will ask the user for a nickname of 10 characters (3c)	I will play the game and check if the program asks me for a nickname	none	none

21. The program will output text which lets the user know what question they're on, the actual question and then the four answers the user can choose from (3d)	I will play the game and check if the program outputs text letting the user know what question they're on, the question and then the four answers they can choose from	none	none
22. The program will output 4 buttons labelled A, B, C and D which the four possible answer are assigned to. This stops the users submitting an invalid answer (3e)	I will play the game and check if the program outputs the buttons	none	none
23. The program will output the closing message containing the users score (3f)	I will play the game until I finish it or get a question wrong to see if I get a closing message	none	none
24. The program will output the sorted leaderboard, showing the players position, nickname and score (3g)	I will view the leaderboard and see if the position, nickname and score are displayed	none	none

Test	Evidence
1.	

2.

```
**********************************************************************

                  Lucy's Big Quiz

**********************************************************************
Please choose one of the following options:

1. Play Game
2. Rules
3. Leaderboard
4. Exit
2
**********************************************************************
               Rules

**********************************************************************
Lucy's Big Quiz is a series of multiple-choice questions that each have four possible answers (A,B,C,D). There will be buttons with each letter and you have to press the one you think is the co

You must answer 15 of these questions correctly, one at a time, in order to win.

As soon as you answer a question incorrectly, the game is over.


Enter 1 to return to home page.
```

3.

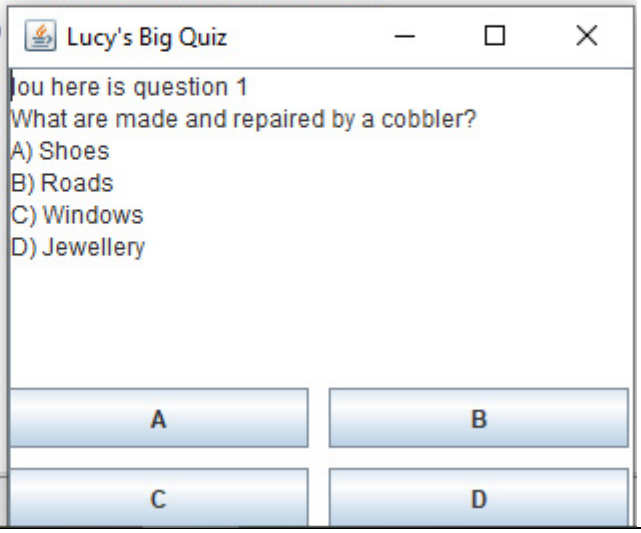
```
**********************************************************************************

                        Lucy's Big Quiz

**********************************************************************************
Please choose one of the following options:

1. Play Game
2. Rules
3. Leaderboard
4. Exit
3
1                       Tim                     15
2                       Sue                     14
3                       Anne                    11
4                       Dan                     8
5                       Greg                    2
```

4.

```
**********************************************************************************

                        Lucy's Big Quiz

**********************************************************************************
Please choose one of the following options:

1. Play Game
2. Rules
3. Leaderboard
4. Exit
4
BUILD SUCCESSFUL (total time: 2 seconds)
```

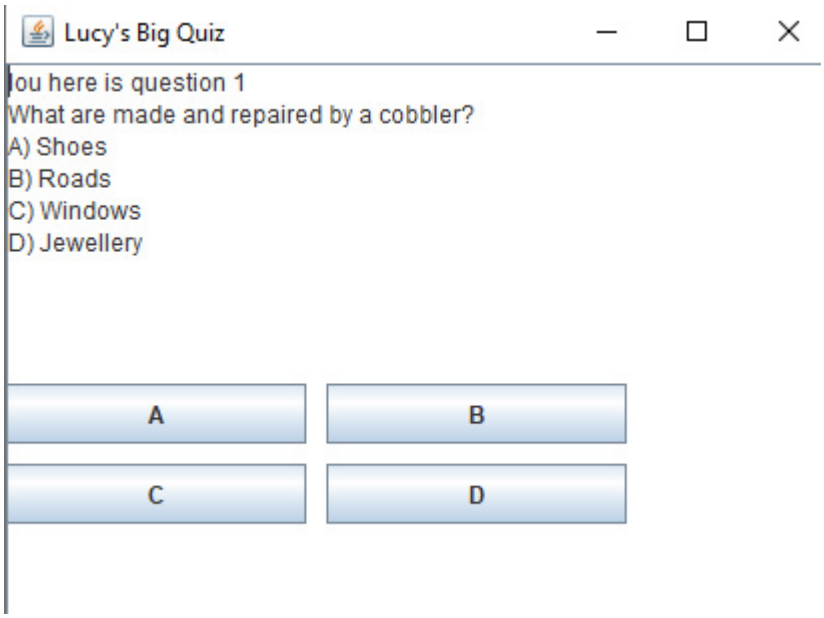
5.

```
**********************************************************************************

                    Lucy's Big Quiz

**********************************************************************************
Please enter your nickname: (10 characters max)
lou
```

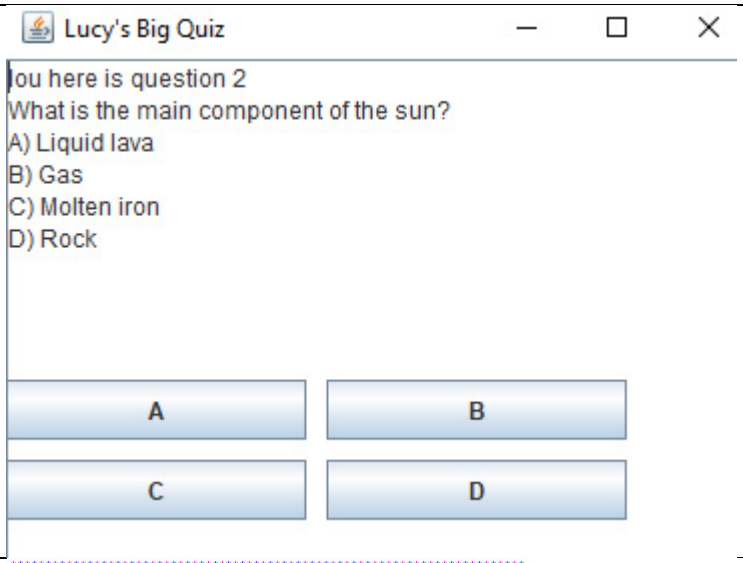


Output

6. Before:



After:



7.

```
**********************************************************************
                Rules
**********************************************************************
Lucy's Big Quiz is a series of multiple-choice questions that each have four possible answers (A,B,C,D). There will be buttons with each letter and you have to press the one you think is the c

You must answer 15 of these questions correctly, one at a time, in order to win.

As soon as you answer a questicn incorrectly, the game is over.


Enter 1 to return to home page.
1
**********************************************************************

                    Lucy's Big Quiz

**********************************************************************
Please choose one of the following options:

1. Play Game
2. Rules
3. Leaderboard
4. Exit
```

8. These are all questions from the data file:

Jou here is question 1
What restaurant franchise advises you to "Eat Fresh"?
A) Taco Bell
B) Subway
C) McDonald's
D) KFC

Jou here is question 2
What is the largest canyon in the world?
A) Verdon Gorge in France
B) King's Canyon in Australia
C) Grand Canyon in the USA
D) Copper Canyon in Mexico

Jou here is question 3
What is the smallest country in the world?
A) Iceland
B) Seychelles
C) Spain
D) Vatican City

9. First play:

Iou here is question 1
What are made and repaired by a cobbler?
A) Shoes
B) Roads
C) Windows
D) Jewellery

A | B
C | D

Iou here is question 2
In the medical profession what do the initials 'GP' stand for?
A) Good Practitioner
B) Garden Practitioner
C) General Practitioner
D) Graded Practitioner

A | B
C | D

Iou here is question 3
Which of these cities is closest to London UK?
A) New York NY
B) Atlanta GA
C) Miami FL
D) Boston MA

A | B
C | D

Jou here is question 4
In what year was the first iPhone released?
A) 2000
B) 2004
C) 2007
D) 2009

A B

C D

Second play:

Jou here is question 1
According to a well-known proverb what do "many hands make"?
A) Many fingers
B) Loud applause
C) Crowded mittens
D) Light work

A B

C D

Jou here is question 2
Which of these chess figures is closely related to 'Bohemian Rhapsody'?
A) Queen
B) King
C) Pawn
D) Bishop

A B

C D

Iou here is question 3
When was the book 'To Kill a Mockingbird' by Harper Lee published?
A) 1950
B) 1960
C) 1970
D) 1980

A | B
C | D

Iou here is question 4
Which former Beatle narrated the TV adventures of Thomas the Tank Engine?
A) John Lennon
B) Paul McCartney
C) George Harrison
D) Ringo Starr

A | B
C | D

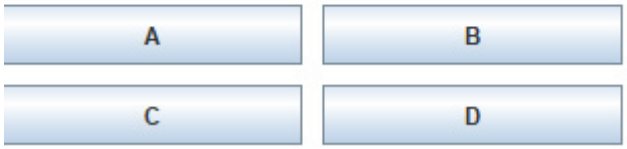
Third play:

Iou here is question 1
Since 2011 Brendan O'Carroll has played the title character in what sitcom?
A) Mrs Brown's Baboons
B) Mrs Brown's Boys
C) Mrs Brown's Babes
D) Mrs Brown's Bust Ups

A | B
C | D

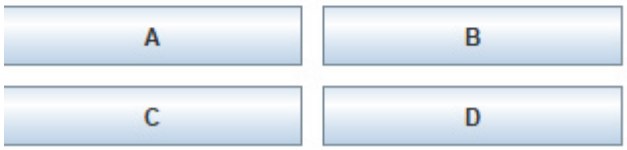
	Jou here is question 2 In the medical profession what do the initials 'GP' stand for? A) Good Practitioner B) Garden Practitioner C) General Practitioner D) Graded Practitioner A B C D Jou here is question 3 Which of these cities is closest to London UK? A) New York NY B) Atlanta GA C) Miami FL D) Boston MA A B C D Jou here is question 4 Where did the powers of Spiderman come from? A) He was born with them B) He was bitten by a radioactive spider C) He went through a scientific experiment D) He woke up with them after a dream A B C D
10.	**When I got the question right:**

lou here is question 1
Since 2011 Brendan O'Carroll has played the title character in what sitcom?
A) Mrs Brown's Baboons
B) Mrs Brown's Boys
C) Mrs Brown's Babes
D) Mrs Brown's Bust Ups



It moved on to question 2.

lou here is question 2
What type of powder is typically contained in a powder keg?
A) Baking powder
B) Talcum powder
C) Foot powder
D) Gunpowder



When I got Q2 wrong:

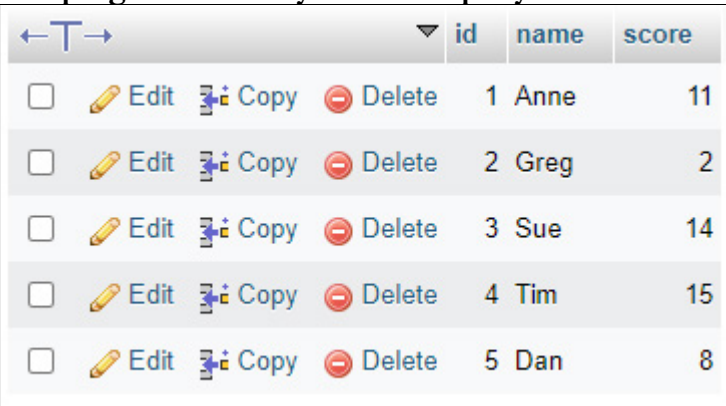
```
Incorrect answer. Better luck next time
You scored 1/15
1                   Tim                 15
2                   Sue                 14
3                   Anne                11
4                   Dan                 8
5                   Greg                2
```

11. **I correctly answered 6 questions and this was displayed:**

```
Incorrect answer. Better luck next time
You scored 6/15
```

The program correctly counted up my correct answers

12.



		id	name	score
☐	Edit Copy Delete	1	Anne	11
☐	Edit Copy Delete	2	Greg	2
☐	Edit Copy Delete	3	Sue	14
☐	Edit Copy Delete	4	Tim	15
☐	Edit Copy Delete	5	Dan	8

13. The top 5 scores are displayed:

	1 Tim 15 2 Sue 14 3 Anne 11 4 Dan 8 5 Greg 2
14.	**I was sorted into the right place through the bubble sort:** You scored 11/15 1 Tim 15 2 Sue 14 3 Anne 11 4 lou 11 5 Dan 8
15.	Enter 1 to return to home page. 7 Wrong input. Please enter 1 9 Wrong input. Please enter 1 1 ********** Lucy's Big Quiz ********** Please choose one of the following options: 1. Play Game 2. Rules 3. Leaderboard 4. Exit
16.	********** Lucy's Big Quiz ********** Please choose one of the following options: 1. Play Game 2. Rules 3. Leaderboard 4. Exit 6 Error. Please enter a valid option: 8 Error. Please enter a valid option: 346767564 Error. Please enter a valid option: 2 ********** Rules ********** Lucy's Big Quiz is a series of multiple-choice questions that each have four possible answers (A,B,C,D). There will be buttons with each letter and ycu have to press the one you think is the c You must answer 15 of these questions correctly, one at a time, in order to win. As soon as you answer a question incorrectly, the game is over. Enter 1 to return to home page.
17.	********** Lucy's Big Quiz ********** Please enter your nickname: (10 characters max) armadillomargherita Your nickname is too long. Please enter your nickname: (10 characters max)

18.

```
**********************************************************************************

                        Lucy's Big Quiz

**********************************************************************************
Please choose one of the following options:

1. Play Game
2. Rules
3. Leaderboard
4. Exit
```

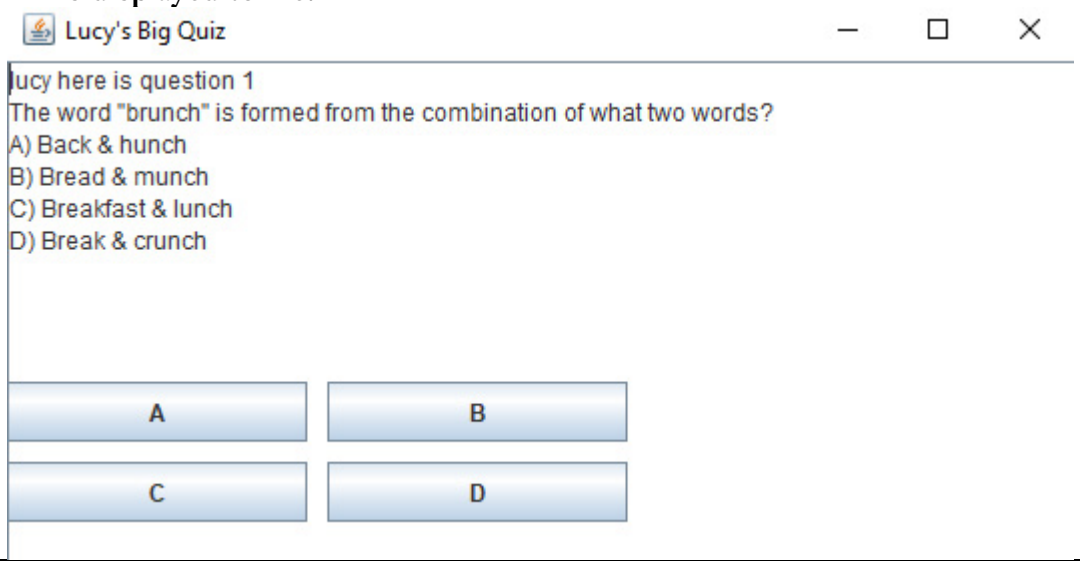
19.

```
**********
Rules
**********
Lucy's Big Quiz is a series of multiple-choice questions that each have four possible answers (A,B,C,D). There will be buttons with each letter and you have to press the one you think is the c

You must answer 15 of these questions correctly, one at a time, in order to win.

As soon as you answer a question incorrectly, the game is over.

Enter 1 to return to home page.
```

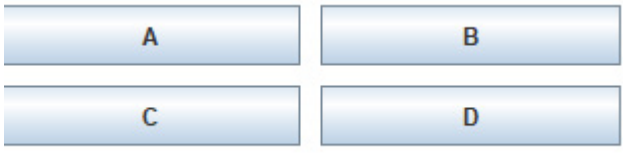
20. **The program asks me for a nickname:**

```
**********************************************************************************

                        Lucy's Big Quiz

**********************************************************************************
Please enter your nickname: (10 characters max)
```

21. **All is displayed to me:**



22. **The buttons are displayed:**



23.

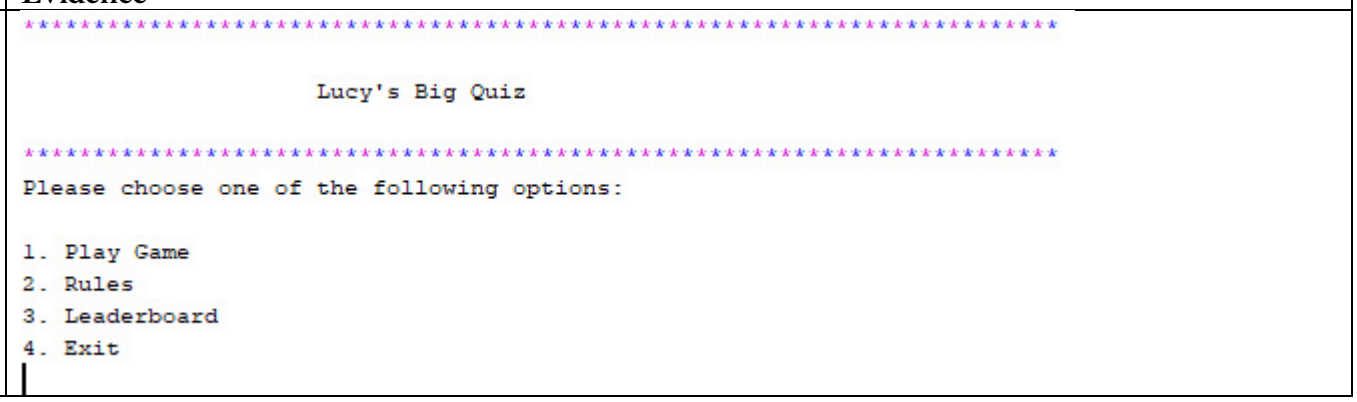
```
Incorrect answer. Better luck next time
You scored 2/15
```

24. The position, name and score are displayed:

	1	Tim	15
	2	Sue	14
	3	Anne	11
	4	Dan	8
	5	Greg	2

4.3 End-User Requirements Testing

End-User Requirements	description of the test	discovered errors	solution if required
1. There should be an opening message at the beginning (4a)	I will run the program and see what appears at the beginning	none	none
2. The user should enter a nickname rather than their real name (4b)	I will play the game until it asks me for a nickname	none	none
3. The user should have 15 questions to answer (4c)	I will play the game up until the very last question which should be 15	none	none
4. The user should have 4 different answer options provided to them (4d)	I will play the game and check if I have 4 answer options	none	none
5. The user should have buttons to input their answer instead of having to type it themselves which will validate the input (4e)	I will check if there are buttons	none	none
6. The user should lose the game instantly if they get an answer wrong (4f)	I will answer a question wrong on purpose to see if the game stops or continues	none	none
7. There should be a closing message at the end of the program (4g)	I will play until I complete the quiz or answer a question incorrectly and see if a closing message is displayed	none	none
8. The user should be able to view the top 5 names and scores in a leaderboard (4h)	I will view the leaderboard and see what displays	none	none

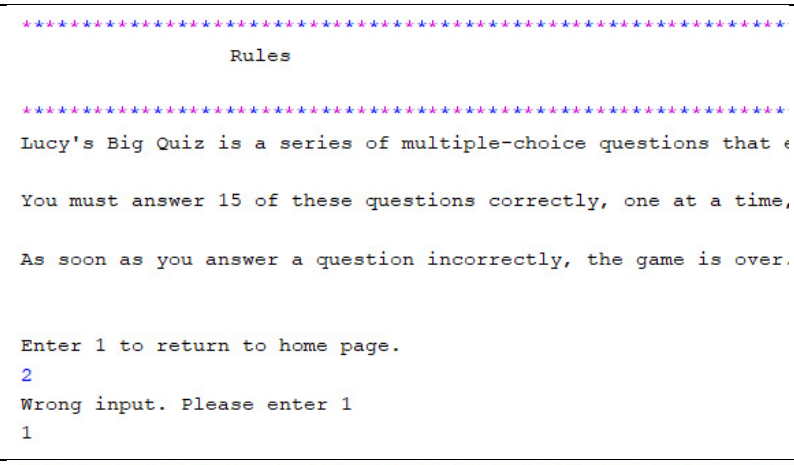
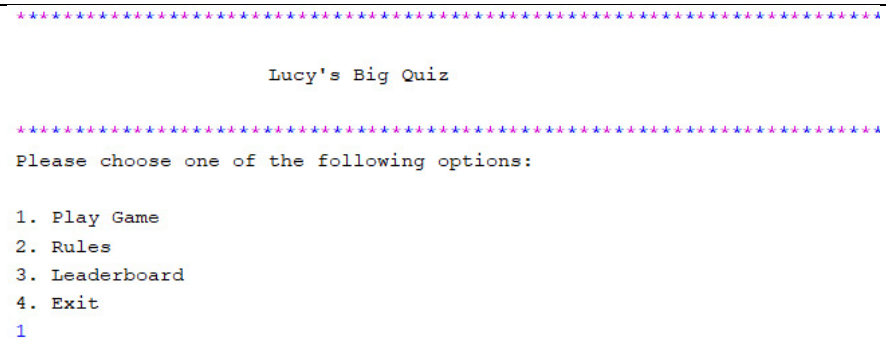
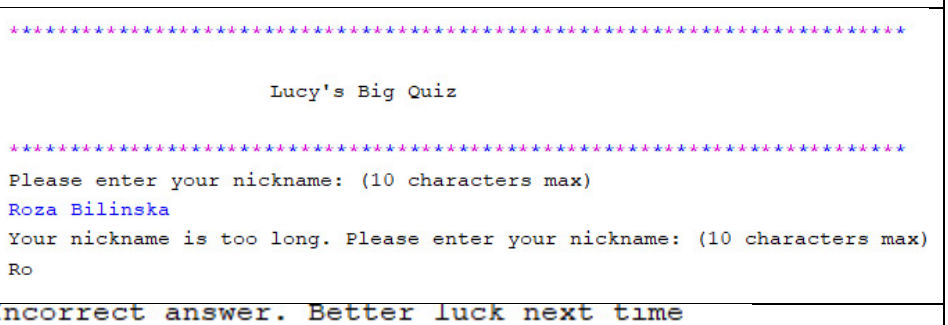
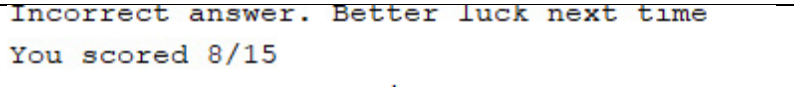
Test	Evidence
1.	

2.	**While there's nothing the program can do to stop the user from entering their real name, the program does ask for a nickname:** ********** Lucy's Big Quiz ********** Please enter your nickname: (10 characters max)
3.	**There are 15 questions:** lou here is question 15 Which creatures suffer from Isle of Wight disease? A) Chickens B) Bees C) Horses D) Fish A B C D lou You scored 15/15
4.	**The user is provided with 4 answer options:** A) Ashton-under-Lyne B) Berwick-upon-Tweed C) Newcastle-upon-Tyne D) Stratford-upon-Avon
5.	**The user has 4 buttons and cannot input an answer through any other way:** A B C D
6.	**I answered the first question incorrectly and the program instantly stopped running:** Incorrect answer. Better luck next time You scored 0/15
7.	**A closing message is displayed:** Incorrect answer. Better luck next time You scored 3/15
8.	1 Tim 15 2 Sue 14 3 Anne 11 4 Dan 8 5 Greg 2

4.4 Persona Testing

Test Case	Task for Personas
1.	View the rules – The user should view the rules and then go back to the home page
2.	Play game – The user should then select to play the game
3.	Input nickname – The user will be asked to input their nickname
4.	The user should answer as many questions as they can
5.	The user should check their score at the end and see their place on the leaderboard if they made it into the top 5

Roza Bilinska

Test Case	Results	Evidence
1.	Roza found it relatively easy to access to rules, however was not aware at first that she had to hit enter to progress. When trying to return to the home page she accidentally pressed the wrong button but the error message allowed her to enter the right option.	
2.	Roza understood that she had to enter 1 to play the game and was able to easily do so.	
3.	When asked to enter a nickname, Roza tried to enter her full name which was too long as there is a maximum of 10 characters but she was able to re-enter a suitable name.	
4.	Roza found the buttons easy to navigate as she was able to just click on them. Roza also liked that the questions were multiple choice which	

	helped her if she was struggling. Roza was able to answer 8 questions correctly.	
5.	Roza was clearly able to see that she had made it to the top 5 on the leaderboard. She could see her name and score.	1 Tim 15 2 Sue 14 3 Anne 11 4 Dan 8 5 Ro 8

Ellis Ramsay

Test Case	**Results** Ellis was	**Evidence**
1.	quickly able to navigate the rules and then back to the home page with no issues.	Lucy's Big Quiz ********** Please choose one of the following options: 1. Play Game 2. Rules 3. Leaderboard 4. Exit 2 ********** Rules ********** Lucy's Big Quiz is a series of multiple-choice questions that eac You must answer 15 of these questions correctly, one at a time, i As soon as you answer a question incorrectly, the game is over. Enter 1 to return to home page. 1 ********** Lucy's Big Quiz ********** Please choose one of the following options: 1. Play Game 2. Rules 3. Leaderboard 4. Exit

2.	Ellis had no issues getting to playing the game	********** Lucy's Big Quiz ********** Please choose one of the following options: 1. Play Game 2. Rules 3. Leaderboard 4. Exit 1
3.	Ellis entered her nickname which met the input validation of 10 characters max and was also not her real name	********** Lucy's Big Quiz ********** Please enter your nickname: (10 characters max) Eli
4.	Ellis was able to score 13 in the quiz. She commented on how she liked the buttons as it made the process faster since she did not have to type her answers. She also said some questions were a bit difficult to answer.	Incorrect answer. Better luck next time You scored 13/15
5.	Ellis said the leaderboard was clear and easy to read and that she could see that she was on it.	1 Tim 15 2 Sue 14 3 Eli 13 4 Anne 11 5 Dan 8

Evaluation

5.1 Fitness for Purpose

Functional Requirement	Requirement met?
(1a) The user can start playing the game by inputting 1 from the keyboard	✓
(1b) The user can view the rules by inputting 2 from the keyboard	✓
(1c) The user can view the leaderboard by inputting 3 from the keyboard	✓
(1d) The user can exit the program by inputting 4 from the keyboard	✓
(1e) The user will enter their username	✓
(1f) The user will input their answer by selecting one of the buttons shown to them	✓
(1g) When provided an option to go back to the home page the user should input 1 from the keyboard	✓
(2a) The program will read in the questions from the data file	✓
(2b) The program will randomise one question from the first set of 10 questions which are of the easiest difficulty. As the user progresses the program will move on to the next set of 10 questions and randomly pick one from there	✓
(2c) When the user selects the button the program will check if the answer they selected matches up with the question. If it does the program moves on to the next question and if it is wrong then the game stops and the user is shown their score	✓
(2d) The program will count up how many questions the user got correct	✓
(2e) The user's username and score will be stored in a table in an external database	✓
(2f) A leaderboard will be created containing the top 5 highest scores	✓
(2g) A bubble sort will sort the leaderboard into the highest scores	✓
(2h) When provided an option to go back to the home page the program should validate whether the user inputs 1	✓
(2i) On the home page the program should validate the users input and check if its 1, 2, 3 or 4	✓
(2j) The program should validate the users nickname to make sure it is no more than 10 characters	✓
(3a) The program will display a message at the start of the game with the name of the game and the options the user can pick	✓
(3b) If the user selects 'rules' then the program will display the rules	✓
(3c) The program will ask the user for a nickname of 10 characters	✓
(3d) The program will output text which lets the user know what question they're on, the actual question and then the four answers the user can choose from	✓
(3e) The program will output 4 buttons labelled A, B, C and D which the four possible answer are assigned to. This stops the users submitting an invalid answer	✓
(3f) The program will output the closing message containing the users score	✓
(3g) The program will output the sorted leaderboard, showing the players position, nickname and score	✓

End-User Requirement	Requirement met?
(4a) There should be an opening message at the beginning	✓
(4b) The user should enter a nickname rather than their real name	✓
(4c) The user should have 15 questions to answer	✓
(4d) The user should have 4 different answer options provided to them	✓
(4e) The user should have buttons to input their answer instead of having to type it themselves which will validate the input	✓
(4f) The user should lose the game instantly if they get an answer wrong	✓
(4g) There should be a closing message at the end of the program	✓
(4h) The user should be able to view the top 5 names and scores in a leaderboard	✓

I would say my program is fit for purpose. It meets all of the functional requirements and the end-user requirements specified at the beginning of my project. The user is able to play my quiz game of 15 questions with the use of JButtons which I think worked really well with my project. What also went well is that all my inputs are validated, however something I think I could work on in the future is validating wrong data types. In terms of testing, I feel that everything went well. There was nothing that stuck out to me as being broken or missing within my program which leads me to believe that I followed the requirements very well and through my ongoing testing and making sure all inputs were validated I was able to catch things before they became an issue.

5.2 Maintainability and Robustness

Maintainability

I have carried out several techniques in order to make my program maintainable. Some of these include leaving commentary in my code, using modularity, having meaningful variables name and also white space. All of these make it so that if someone else was to access my program it would be easy to understand and edit without having to spend excessive amounts of time dedicated to understanding what my program does.
I implemented commentary to my code because it makes it easier for not only me but other people to understand what certain lines of code are doing. I also found commentary helpful when it came to labelling my functional and end-user requirements within my code, as well as telling me where certain statements or modules end which is useful as sometimes brackets can get confusing.

```
switch (vOption) {
    case 1: //if the user inputs 1 then the program will read in the questions, then play the game (1a)
        readQuestions(questionsTable);
        playGame(questionsTable, aLeaderboard);
        break;
    case 2: //if the user inputs 2 then the program will show the rules (1b)
        rules();
        break;
    case 3: //if the user inputs 3 then the program will read in the leaderboard data, put it through a bubble sort and then display it (1c)
        readLeaderboard(aLeaderboard);
        sortLeaderboard(aLeaderboard);
        displayLeaderboard(aLeaderboard);
        break;
    case 4: //if the user inputs 4 they will exit the program (1d)
        System.exit(status: 0);
        break;
    } //end switch case
} // end processGame
```

Modularity has made my program maintainable because each module contains code that is quite similar and in most parts work together for a certain part of the program which makes it easier for someone reading my code to view relevant code grouped together. My modules are also organized neatly and in an order of which modules would be called first to last.

```
displayQuestion(questionsTable, aLeaderboard);
```

I have used several meaningful variable names which makes my program maintainable because you can easily tell where that variable is being used and what for. For example, using questionNum instead of just num makes it easy for someone reading the code to identify that this variable must be the number of question the player is on.

```
String playerName = "";
int questionNum = 1;
int vRndNum = 0;
int numCorrect = 0;
```

Whitespace and nesting helps make my program maintainable because it is so much easier to read and whoever is reading the code can separate different modules, statements and also just lines of code that are not similar. Nesting is useful because it makes it easier to see what lines of code are in a particular If statement, for example, whereas without it it's a lot harder to locate.

```
int n = 6; // DECLARE n INITIALLY 6

boolean swapped = true; // DECLARE swapped INITIALLY TRUE

   while ((swapped) && (n > 0)) { // WHILE swapped AND n > 0

    swapped = false; // SET swapped TO False

       for (int i = 0; i < n - 1; i++) { // FOR i = 0 to n-1 DO

        int vNum1 = Integer.parseInt(aLeaderboard[i][2]);
        int vNum2 = Integer.parseInt(aLeaderboard[i+1][2]);
```

Robustness

Robustness regards how my program handles inputs that are invalid. Overall my program is robust as I think it handles invalid inputs well and often states that the users input is not what it is looking for and gives them chances to rectify it.

In terms of the home page, the program is looking for an input of 1, 2, 3 or 4. To test this I will enter any number except these ones. In response, the program will tell the user there is an error and allow them to try again and enter a different number. The program will repeat itself as often as it needs to until the users input meets its requirements.

```
**********************************************************

                    Lucy's Big Quiz

**********************************************************
Please choose one of the following options:

1. Play Game
2. Rules
3. Leaderboard
4. Exit
87
Error. Please enter a valid option:
98
Error. Please enter a valid option:
556
Error. Please enter a valid option:
6
Error. Please enter a valid option:
```

In some cases the user can input 1 to return to the home page. Similar to the home page's input validation, the program will inform the user if their input is not what it is looking for and will allow them to re-enter an input, repeating itself as often as required.

```
Enter 1 to return to home page.
5
Wrong input. Please enter 1
687
Wrong input. Please enter 1
86
Wrong input. Please enter 1
56
Wrong input. Please enter 1
2
Wrong input. Please enter 1
```

The users nickname is a maximum of 10 characters. If the users input exceeds this, the program will ask them to re-enter a nickname of 10 characters or less. Again, the program will repeat itself as long as the conditions are not met.

```
Please enter your nickname: (10 characters max)
qwertyabcdefg
Your nickname is too long. Please enter your nickname: (10 characters max)
abcdefghijklmnop
Your nickname is too long. Please enter your nickname: (10 characters max)
lucylucylucylucylucy
Your nickname is too long. Please enter your nickname: (10 characters max)
```